NFSU

National Forensic Sciences University
Knowledge | Wisdom | Fulfilment
An Institution of National Importance
(Ministry of Home Affairs, Government of India)

PROJECT REPORT

ON

"A Hybrid Approach to DNS Security: Combining Machine Learning-Based Threat Detection with Configuration Analysis and Remediation Guidance"

Submitted To

School of Cybersecurity & Digital Forensics
National Forensic Sciences University

For partial fulfilment for the award of degree

B.Tech.-M.Tech. Computer Science & Engineering

In

Cybersecurity

Submitted By

Rupam Barui
(102CTBMCS2122002)

Under the Supervision of

Mr. Aditya Pratap Singh
Assistant Professor
School of Cybersecurity & Digital Forensics
National Forensic Sciences University

National Forensic Sciences University,
Delhi Campus, New Delhi – 110085, India

April 2026

NFSU

National Forensic Sciences University

Knowledge | Wisdom | Fulfilment

An Institution of National Importance
(Ministry of Home Affairs, Government of India)

PROJECT REPORT

ON

"A Hybrid Approach to DNS Security: Combining Machine Learning-Based Threat Detection with Configuration Analysis and Remediation Guidance"

Submitted To

School of Cybersecurity & Digital Forensics

National Forensic Sciences University

For partial fulfilment for the award of degree

B.Tech.-M.Tech. Computer Science & Engineering

In

Cybersecurity

Submitted By

Rupam Barui

(102CTBMCS2122002)

Under the Supervision of

Mr. Aditya Pratap Singh

Assistant Professor

School of Cybersecurity & Digital Forensics

National Forensic Sciences University

National Forensic Sciences University,

Delhi Campus, New Delhi – 110085, India

April 2026

DECLARATION

I hereby declare that the thesis entitled "**A Hybrid Approach to DNS Security: Combining Machine Learning-Based Threat Detection with Configuration Analysis and Remediation Guidance**" is a research work done by me and no part of the thesis has been presented earlier and will be presented for any degree, diploma or similar title at any other institute/university.

Date: April 26, 2026

Place: New Delhi, Delhi, India

Rupam Barui
102CTBMCS2122002
B.Tech.-M.Tech. Computer Science & Engineering
2026

ORIGINALITY REPORT CERTIFICATE

I certify that

a. The work contained in the dissertation is original and has been done by myself under the supervision of my supervisor.

b. The work has not been submitted to any other Institute for any degree or diploma.

c. I have conformed to the norms and guidelines given in the Ethical Code of Conduct of the Institute.

d. Whenever I have used materials (data, theoretical analysis, and text) from other sources, I have given due credit to them by citing them in the text of the dissertation and giving their details in the references.

e. Whenever I have quoted written materials from other sources, due credit is given to the sources by citing them.

f. From the plagiarism test, it is found that the similarity index of the whole dissertation is within 10% and single paper is less than 10% as per the university guidelines.

Date: April 26, 2026

Place: New Delhi, Delhi, India

Rupam Barui

Enroll. No.: 102CTBMCS2122002

Forwarded by

(Mr. Aditya Pratap Singh)

Date: April 26, 2026

CERTIFICATE

This is to certify that the work contained in the dissertation entitled "**A Hybrid Approach to DNS Security: Combining Machine Learning-Based Threat Detection with Configuration Analysis and Remediation Guidance**", submitted by **Rupam Barui** holding the enrollment number: **102CTBMCS2122002** for the award of the degree of **B.Tech.-M.Tech. Computer Science & Engineering** in Cybersecurity to the **National Forensic Sciences University**, **Delhi Campus**, is a record of bonafide research works carried out by him under my supervision and guidance.

Date: April 26, 2026
Place: New Delhi, Delhi, India

Mr. Aditya Pratap Singh
Assistant Professor
School of Cybersecurity & Digital Forensics
National Forensic Sciences University
Delhi Campus, New Delhi – 110085, India

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to all those who have contributed to the successful completion of this dissertation.

First and foremost, I am deeply grateful to my dissertation supervisor, **Mr. Aditya Pratap Singh**, Assistant Professor, School of Cybersecurity & Digital Forensics, National Forensic Sciences University, for his invaluable guidance, constant encouragement, and constructive feedback throughout the course of this research. His expertise and insights have been instrumental in shaping this work.

I extend my heartfelt thanks to the **Head of the Department** and all the **faculty members** of the School of Cybersecurity & Digital Forensics for their academic support and for providing a stimulating research environment. Their teachings have laid a strong foundation for this work.

I am also thankful to my **lab mates and peers** for their cooperation, technical discussions, and moral support during the challenging phases of this project. The collaborative spirit within the department has greatly enriched my learning experience.

I would like to acknowledge the **National Forensic Sciences University, Delhi Campus**, for providing the necessary infrastructure, resources, and facilities that made this research possible.

Finally, I owe my deepest gratitude to my **family** for their unwavering support, patience, and encouragement throughout my academic journey. Their belief in me has been a constant source of motivation.

With Sincere Regards,

Rupam Barui
102CTBMCS2122002
B.Tech.-M.Tech. Computer Science &
Engineering
2026

ABSTRACT

The Domain Name System (DNS) is one of the core pillars of the modern internet but it is a frequent target for exploitation, misconfiguration, and misuse. In this dissertation, we introduce **DNS Guard**, a hybrid domain threat intelligence system that integrates ML-based threat detection with deep DNS configuration analysis and automated remediation guidance. We evaluate its performance as an automated tool for detecting domains involved in various threats including malicious, misconfigured and suspicious entities.

Our system generates a 40-feature vector per domain including lexical, structure based, DNSSEC validation and composite security metrics. A Random Forest classifier combined with a Multi-Layer Perceptron (MLP) neural network classifier provides a 0–100 threat score along with SHAP-style feature importance explanations to increase transparency of the decision making. Additionally, the system integrates several intelligence modules: WHOIS lookup with change tracking, Certificate Transparency logs analysis, passive DNS history with fast-flux detection, typosquatting detection for top world brands, subdomain enumeration, reverse IP lookups, real-time blocklist checking with Spamhaus DBL and PhishTank, andIP geolocation plotting onto an interactive SVG world map. Full report in PDF is automatically generated on demand.

Our platform is complemented by a context-aware AI chatbot that leverages the Groq API (LLaMA 3.1 8B Instant) or an embedded rule-based system for key DNS security domains. It includes a RAG pipeline leveraging a Qdrant vector database to facilitate the AI model in providing relevant security guidance for each specific domain. The system has a containerized microservices architecture, developed with Docker Compose using a Next.js 14 frontend, FastAPI with Celery on the backend and an independent ML service, PostgreSQL as the database and Redis as a message broker. We demonstrate that our system accurately identifies malicious, misconfigured and suspicious domains, serving as an extensible tool for DNS security monitoring and threat intelligence.

Keywords: DNS Security, Threat Intelligence, Machine Learning, DNSSEC, Typosquatting, Fast-Flux Detection, Phishing Detection, Natural Language Processing, RAG, Cybersecurity.

Contents

Declaration **i**

Originality Report Certificate **ii**

Certificate **iii**

Acknowledgement **iv**

Abstract **v**

Abbreviations **ix**

List of Tables **xi**

List of Figures **xii**

Introduction **1**

1.1 Background 1
1.2 Problem Statement 2
1.3 Objectives 2
1.4 Scope 3
1.5 Report Organisation 4

Literature Survey **5**

2.1 Current / Existing System 5
2.1.1 Study of Current System 5
2.1.2 Problem and Weakness of Current System 6
2.2 Requirements of New System 6
2.3 Feasibility Study 8
2.3.1 Technical Feasibility 8
2.3.2 Operational Feasibility 8
2.4 Tools and Technology Required 8
2.4.1 Frontend: Next.js 14 and Tailwind CSS 9
2.4.2 Backend: FastAPI and Celery 9

2.4.3 Database and Storage: PostgreSQL, Redis, and Qdrant 9
2.4.4 Machine Learning: scikit-learn 10
2.4.5 DNS Resolution: dnspython 10
2.4.6 PDF Generation: ReportLab 11
2.4.7 AI and LLM: Groq API 11
2.4.8 Containerisation: Docker and Docker Compose 11

Design: Analysis, Design Methodology and Implementation Strategy **12**
3.1 Function of System 12
3.1.1 Use Case Diagram 12
3.1.2 Activity Diagram 13
3.1.3 Sequence Diagram 14
3.2 Data Modelling 15
3.2.1 Entity-Relationship Diagram 15
3.2.2 Class Diagram 16
3.3 Functional and Behavioural Modelling 18
3.3.1 Data Flow Diagram 18
3.3.2 Data Dictionary 19

Implementation **23**
4.1 Implementation Environment 23
4.1.1 Model Used in Developing 23
4.1.2 Software Prototyping Types 24
4.2 Coding Standard 24
4.3 Laboratory Setup 25
4.4 Tools and Technology Used 25
4.5 Screenshots / Snapshots 26

Summary of Results and Future Scope **29**
5.1 Advantages and Unique Features 29
5.2 Results and Discussions 30
5.3 Future Scope of Work 30

Conclusion **32**

References **34**

A API Reference **I**

B Machine Learning Model Details **III**

C Sample Scan Output **VI**

D Environment Variables and Configuration **VIII**

E Installation and Deployment Guide **X**

F Sample Generated PDF Report **XII**

List of Publications / Online References **XVI**

List of Abbreviations

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
CT	Certificate Transparency
DBL	Domain Block List
DKIM	DomainKeys Identified Mail
DMARC	Domain-based Message Authentication, Reporting and Conformance
DNS	Domain Name System
DNSKEY	DNS Key Record
DNSSEC	Domain Name System Security Extensions
DS	Delegation Signer
FYP	Final Year Project
HTML	HyperText Markup Language
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
IP	Internet Protocol
LLM	Large Language Model
ML	Machine Learning
MLP	Multi-Layer Perceptron
MX	Mail Exchange
NLP	Natural Language Processing
NS	Name Server
PDF	Portable Document Format
RAG	Retrieval-Augmented Generation
RF	Random Forest
RRSIG	Resource Record Signature
SHAP	SHapley Additive exPlanations
SPF	Sender Policy Framework

Abbreviation	Full Form
SSL	Secure Sockets Layer
SVG	Scalable Vector Graphics
TLD	Top-Level Domain
TLS	Transport Layer Security
TXT	Text Record
URL	Uniform Resource Locator
WHOIS	(Protocol for querying domain registration information)

List of Tables

3.2 Data Dictionary 20
4.3 Laboratory Setup 25
4.4 Tools and Technologies Used 26

A.1 API Endpoint Reference I

B.1 ML Model Feature Vector (40 Features) III
B.2 ML Model Evaluation Metrics V

D.1 Environment Variables and Configuration VIII

List of Figures

3.1 Use Case Diagram for DNS Guard 13
3.2 Activity Diagram for Domain Analysis 14
3.3 Sequence Diagram for Domain Analysis Request 15
3.4 Entity-Relationship Diagram for DNS Guard Database 16
3.5 Class Diagram for DNS Guard Backend 17
3.6 Data Flow Diagram – Level 0 (Context Diagram) 18
3.7 Data Flow Diagram – Level 1 19
4.8 DNS Guard – Home Page with Search Interface 26
4.9 DNS Guard – Analysis Dashboard (Part 1: Threat Score and DNS Records) . . 27
4.10 DNS Guard – Analysis Dashboard (Part 2: WHOIS and Certificate Transparency) 27
4.11 DNS Guard – Analysis Dashboard (Part 3: Threat Intelligence and Geolocation) 28
4.12 DNS Guard – Light Mode Interface 28

Introduction

1.1 Background

Today, billions of users, devices, and services around the globe are connected via the internet. The cornerstone of this connection is a system called the Domain Name System (DNS). It functions in a way comparable to a phonebook for the internet; users type in something like `www.example.com` into their web browser, and DNS translates this human-readable form into a string of numbers, an IP address, which the computer uses to find the correct server and make a connection [1]. Without DNS, users would have to learn the series of numbers to visit a website.

Since DNS is so integral to the operation of the internet, it has been and continues to be a lucrative target. Through the years, there have been numerous ways that criminals have exploited DNS to conduct their malicious activities, such as registering very similar domains to popular websites in order to trick users into visiting their site (typosquatting) [2]; using DNS to distribute malware, host phishing sites or hide the location of their command and control server. In order to avoid being shut down, attackers will use methods such as fast-flux DNS to constantly change the IP address associated with their domain [3].

At the same time, many organizations also have improperly configured DNS records. Missing or malformed SPF, DKIM and DMARC records will result in an organization's email system being vulnerable to spoofing attacks [4]. If a domain does not use DNSSEC, it is susceptible to DNS cache poisoning, where a user can be rerouted to a false website without realizing it [5]. These misconfigurations are extremely common and will often not be realized until a attack is successful.

Security researchers and administrators require tools that are able to quickly determine if a domain has any of these issues all in one location. The vast majority of existing tools will only identify one or two of these problems and have clearly identified a need for an all-in-one platform for DNS analysis, threat intelligence, machine learning based scoring, and clear remediation steps.

1.2 Problem Statement

Even though DNS is so vital for internet, there are still no easy-to-use and all-in-one tools which can check a domain from various security aspects simultaneously. Some available solutions are too limited, too expensive for individuals and small companies, or require too much technical know-how for ordinary people to analyze with.

The specific problems this project addresses are as follows:

- DNS abuse, including phishing, typosquatting, and fast-flux attacks, continues to grow every year [6].
- Many domain owners do not know whether their DNS configuration is correct or secure.
- There is no single open tool that combines DNS record analysis, WHOIS data, certificate transparency, passive DNS history, threat intelligence feeds, and machine learning-based risk scoring in one platform.
- When a domain is flagged as suspicious, users often do not receive clear guidance on what the problem is or how to fix it.

DNS Guard was designed to combat this by being a web based application that conducts a comprehensive security check for any given domain and displays the results in an easily understood format.

1.3 Objectives

The main objectives of this project are:

1. To build a platform that can analyse any domain for DNS misconfigurations, security weaknesses, and signs of malicious activity.
2. To develop a machine learning model that assigns a threat score between 0 and 100 to a domain based on its DNS records and lexical features.
3. To integrate multiple threat intelligence sources, including Spamhaus DBL and PhishTank, to check whether a domain appears on known blocklists.
4. To detect typosquatting by comparing a given domain against a list of top global brands using string similarity techniques.
5. To provide WHOIS analysis, certificate transparency monitoring, passive DNS history, subdomain enumeration, and reverse IP lookup as part of a single scan.

6. To include an AI-powered chatbot that can answer questions about DNS security and explain the results of a scan in plain language.
7. To generate a downloadable PDF report summarising all findings for a given domain.
8. To build the system as a set of containerised microservices so that it is easy to deploy and maintain.

1.4 Scope

This project covers the following areas:

- Analysis of domain names and their associated DNS records, including A, MX, NS, TXT, DNSKEY, RRSIG, and DS records.
- WHOIS data retrieval and comparison between scans to detect changes in domain registration.
- Certificate transparency log queries to monitor SSL/TLS certificates issued for a domain.
- Passive DNS history to identify historical IP addresses and detect fast-flux behaviour.
- Typosquatting detection using edit-distance algorithms against a curated list of top brands.
- Subdomain enumeration using both brute-force wordlists and certificate transparency logs.
- Reverse IP lookup to find other domains hosted on the same IP address.
- Threat intelligence checks against Spamhaus DBL and PhishTank.
- IP geolocation and visualisation on a world map.
- A machine learning ensemble model combining a Random Forest classifier and a Multi-Layer Perceptron (MLP) neural network for threat scoring.
- A context-aware chatbot using a local Large Language Model (LLM) with a Retrieval-Augmented Generation (RAG) pipeline.
- PDF report generation for each domain scan.

The project does not cover real-time network traffic analysis, deep packet inspection, or active exploitation testing. It is designed as a passive analysis and intelligence tool.

1.5 Report Organisation

The rest of this report is organised as follows:

- **Chapter 2 – Literature Survey** reviews existing work on DNS security, threat detection, machine learning for cybersecurity, and related tools.
- **Chapter 3 – Design** covers the system architecture, the analysis of requirements, the design methodology, and the implementation strategy.
- **Chapter 4 – Implementation** describes how each component of the system was built, including the frontend, backend, AI service, and supporting infrastructure.
- **Chapter 5 – Summary of Results and Future Scope** presents the results of testing and evaluation, and discusses possible improvements and extensions.
- **Chapter 6 – Conclusion** summarises the work done and the key findings of the project.

Literature Survey

2.1 Current / Existing System

2.1.1 Study of Current System

There are numerous tools and platforms that help security researchers and network administrators assess the security of a domain. Some examples and their specific purpose are presented below:

VirusTotal [7] is one of the most widely used tools. It scans URLs, files, and domains and reports if it has been blacklisted or flagged as malicious by one of over 70 antivirus engines. VirusTotal does not offer advanced DNS features such as the ability to inspect and analyze all of the DNS records associated with a domain or to validate the integrity of those records using DNSSEC, or recommendations on fixing any potential issues in their configuration.

MXToolbox [8] has a number of features and tests related to DNS, but is primarily geared towards analyzing email servers and configurations such as SPF, DKIM and DMARC records and provides debugging assistance with mail issues.

Shodan [9] is a search engine used for scanning the entire internet and displaying a report about connected devices found at a certain IP address. It can give researchers information about the servers associated with a domain, open ports, and software that runs on that device. It is incredibly powerful, but relies more on technical expertise.

SecurityTrails [10] offers historical DNS information, WHOIS information, and information about the subdomains associated with a domain. This service is valuable for understanding passive DNS intelligence and provides limited information for free; in order to receive the complete set of information SecurityTrails is a paid service.

DNSdumpster [11] is a free tool designed to enumerate all of the subdomains associated with a particular domain. It offers no threat intelligence reporting but helps visualize the attack surface of a domain.

PhishTank [12] and **Spamhaus** [13] both maintain lists of domains that are identified as malicious and block them, in a community-based approach. These services often offer these lists for consumption as threat feeds to be incorporated into other tools or services, but are not analysis platforms in themselves.

In the realm of research on DNS threat detection there have been several significant projects

such as Antonakakis et al.'s EXPOSURE system [14] which utilized passive DNS data along with machine learning for detecting malicious domains, and its successor which analyzed more features for greater detection [15]. More recently, machine learning methods have been used to identify domains generated by malware known as DGAs (domain generation algorithms) [16]. However, these research projects often do not provide a user-friendly interface or actionable remediation guidance for domain owners, and they do not integrate the wide range of features that DNS Guard aims to provide in one platform.

2.1.2 Problem and Weakness of Current System

While the tools described above are useful individually, they all have significant limitations when considered as a complete solution for DNS security analysis.

- **Fragmentation:** No single free tool combines DNS record analysis, WHOIS data, certificate transparency, passive DNS, typosquatting detection, subdomain enumeration, reverse IP lookup, and threat intelligence in one place. Users must visit multiple tools and manually combine the results.
- **No unified threat score:** Most tools present raw data without summarising the overall risk level of a domain. This makes it difficult for non-expert users to quickly understand whether a domain is safe or dangerous.
- **No remediation guidance:** Tools like MXToolbox show what is wrong with a DNS configuration but do not explain why it matters or how to fix it in simple terms.
- **Cost:** Many of the most capable tools, such as SecurityTrails and Shodan, require paid subscriptions. This limits access for students, independent researchers, and small organisations.
- **No AI assistance:** None of the existing free tools include a conversational AI assistant that can answer questions about the results of a scan or explain DNS security concepts in plain language.
- **Limited explainability:** Academic machine learning models for DNS threat detection often do not explain which features contributed to a classification decision, making it hard to trust or act on the results [17].

2.2 Requirements of New System

Based on the study of existing systems and their weaknesses, the following requirements were identified for DNS Guard.

Functional Requirements:

- The system must accept any domain name as input and perform a full security analysis.
- It must check DNS records including A, MX, NS, TXT, DNSKEY, RRSIG, and DS.
- It must validate DNSSEC configuration and report whether it is properly set up.
- It must retrieve and display WHOIS data and detect changes between scans.
- It must query certificate transparency logs to find SSL/TLS certificates issued for the domain.
- It must retrieve passive DNS history and detect fast-flux behaviour.
- It must check the domain against Spamhaus DBL and PhishTank blocklists.
- It must detect typosquatting by comparing the domain against a list of top brands.
- It must enumerate subdomains using brute-force and certificate transparency methods.
- It must perform reverse IP lookup to find co-hosted domains.
- It must produce a threat score between 0 and 100 using a machine learning model.
- It must provide feature importance information alongside the threat score.
- It must include an AI chatbot that can answer DNS security questions.
- It must generate a downloadable PDF report for each scan.

Non-Functional Requirements:

- The system must be fast enough to return results within a reasonable time for a typical domain.
- The user interface must be clean and easy to use for both technical and non-technical users.
- The system must support both dark and light display modes.
- The system must be deployable using Docker Compose with minimal setup.
- The system must store scan history in a database for future reference.

2.3 Feasibility Study

2.3.1 Technical Feasibility

All technologies involved in building DNS Guard are mature, documented, and free to use. Python offers a lot of libraries for DNS resolution, WHOIS lookup, ML and Web development. `dnspython` library [18] allows a robust DNS query interface. The `scikit-learn` library [19] provides the necessary building blocks and training mechanisms for the ML model. `FastAPI` [20] is a modern, high-performance Python web framework which suits well for creating APIs. The `Next.js` framework [21] is well suited for building the frontend providing server side rendering and a smooth developer experience.

Packaging all the services into containers and making the system deployable on any platform is simple using `Docker` and `Docker Compose` [22]. Both `PostgreSQL` [23] and `Redis` [24] are well established choices for data storage and queuing system respectively. `Qdrant` [25] is a vector store and perfectly fits the RAG mechanism in the chatbot.

`Groq` [26] allows for high speed inference with the LPU architecture and even offers a generous free tier. `LLaMA 3.1 8B Instant` is a powerful open source LLM which produces high quality replies with low latency.

To summarize, the project is highly feasible as all the tools and services used are well-established and tested in various similar applications.

2.3.2 Operational Feasibility

The use of docker images makes deployment and use of DNS Guard as simple as a single `docker compose up` command. This removes the need to install or set up any additional components within your environment. Accessing the web UI requires no special software, only a web browser.

The target audience of DNS Guard are researchers, network administrators and students. The interface aims to clearly and concisely provide results. Results displayed through color-coded threat scores and the ability to geolocate IPs with the world map are both a visual help. The AI chatbot aims to help anyone that may not understand technicalDNS Security terms.

Operationally, DNS Guard could be utilized by a small organization or academic institution. High volume usage will require some adjustments and more work on the automatic updates of the threat intelligence feed.

2.4 Tools and Technology Required

This section describes each tool and technology used in DNS Guard, explains what it does, why it was chosen over alternatives, and how it fits into the overall system.

2.4.1 Frontend: Next.js 14 and Tailwind CSS

Next.js 14 [21] is a React based framework from Vercel and was chosen to implement the frontend for DNS Guard for a few reasons. Firstly, support for server-side rendering and for the App router was required as the pages are rendered on the server, to then be sent to the browser, so that the website pages can load faster and be more responsive. Secondly, there is in-built support for API routes within Next.js. These were used to implement the proxy endpoints that forward requests from the browser to the backend without giving the backend's URL directly to the user. Finally, the Next.js community is large, and it has good documentation and usage within production applications so it seemed like a good, safe bet to use for this project.

Another possible option would have been plain React along with an Express.js server, but this would have required more configuration and it wouldn't have come with as many built in optimizations.

Tailwind CSS [27] is a utility-first CSS framework. Instead of writing custom CSS classes and applying them to HTML elements you simply use tiny, single-purpose utility classes directly in the HTML. This was chosen for speed and also ease of use when trying to build a responsive and cohesive design, and also comes with support for dark mode which was utilized in DNS Guard. Writing CSS classes from scratch would have taken longer and be less easy to maintain.

2.4.2 Backend: FastAPI and Celery

FastAPI [20] is my chosen as my web framework as it is a modern, fast web framework that can be used for both the primary backend and the AI service, and can automatically create an interactive API documentation. FastAPI relies heavily on Python type hints to validate API input and output, leading to fewer bugs and more readable code. It is considerably faster and provides a much better way to handle async code than alternative Python frameworks like Flask, a feature that proved essential when handling the multiple simultaneous DNS queries and calls to external services.

Celery [28] is a distributed task queue that works with Python. Many of the steps required to perform DNS analysis, including the resolution, WHOIS, CT queries and ML scoring, would be far too slow to be processed in a synchronous HTTP request without causing a timeout. Instead, Celery is used to allow these steps to run asynchronously in the background by worker processes, while the frontend polls for results. The message broker used by Celery is Redis, which passes messages between the API and the workers.

2.4.3 Database and Storage: PostgreSQL, Redis, and Qdrant

PostgreSQL [23] is a robust open source relational database management system. It was selected for the storage of the scan results, the WHOIS history and other persistent data because it

is dependable and efficiently handles and queries large volumes of structured data. Although it would have been possible to use the simpler SQLite database, that solution does not allow concurrent access to data, unlike PostgreSQL which is a suitable candidate for a multi-service application where the backend and workers might need to access both read/write both the database simultaneously.

Redis [24] is an in-memory data structure store and has been selected to be the message broker for Celery in the DNS Guard. Upon the submission of a domain by a user the backend places a new Celery task on the queue to be picked up by a Celery worker from Redis. Redis was selected as it is extremely fast, lightweight and by far the most common message broker in use by Celery. Redis has also been utilized to cache commonly accessed data to further improve performance by reducing database requests to a minimum.

Qdrant [25] is a vector search engine optimized for storing and querying high-dimensional vector embeddings. Qdrant is utilized in the DNS Guard to drive the RAG pipeline for the AI chatbot. DNS security documents and knowledge base entries are embedded and stored into the Qdrant database. When the user query the chatbot, the query is also embedded and Qdrant search the most relevant documents that are then passed onto the LLM to enable a correct and informative answer to be generated. Qdrant was preferred to other options such as FAISS due to its operation as a standalone service, support for filtering and integration with a microservices setup.

2.4.4 Machine Learning: scikit-learn

scikit-learn [19] library, implemented in python, is one of the most popular libraries in the machine learning community. Several state-of-the-art algorithms have been developed by this community and some of these have been used in our model. We've specifically used the Random Forest algorithm (one of the two models forming our threat scoring ensemble). The library has been selected because of its comprehensive documentation, ease of implementation, and the ability to deliver consistent results. Other benefits offered by this library and have also been employed in the development of our model include features extraction utilities, model assessment and validation. For our MLP classifier we also utilized the functionality offered by the MLPClassifier from the same library. This gave us a balance between complexity and efficiency unlike the complete deep learning libraries like TensorFlow or PyTorch.

2.4.5 DNS Resolution: dnspython

Dnspython [18] is a DNS query library for Python. We used this in the backend to query for A, MX, NS, TXT, DNSKEY, RRSIG and DS DNS records for the requested domain. It is ideal for this purpose, as it can perform various types of DNS record lookups, handle DNSSEC validation, and it is a well maintained standard choice when it comes to working with DNS records in Python. There are no viable alternatives.

2.4.6 PDF Generation: ReportLab

ReportLab [29] is a Python module to generate PDF documentprogrammatically. ReportLab is utilized for generating the reports available as downloadable PDF's within DNS Guard. It has been selected as the library for ReportLab offers complete control of the layout of the PDF and its content (e.g., tables, charts, formatted text etc.). Another choice would have been to use a headless browser for rending HTML in the browser as PDF's, however this solution is much more heavier and slower than ReportLab; ReportLab generates PDFs by running straight from the Python code.

2.4.7 AI and LLM: Groq API

Groq API [26]is a cloud service for LLM inference providing a lightning-fast response due to the LPU (Language Processing Unit) hardware and was utilized in DNS Guard to provide the functionality of the AI chatbot through the **LLaMA 3.1 8B Instant** model. It had good rate limits on the free plan that are appropriate for academic projects and is OpenAI compatible. It was also implemented to use a rule-based fall-back engine if the API is not accessible or the API key is not set.

2.4.8 Containerisation: Docker and Docker Compose

Docker [22] framework is utilized. Docker is a platform for developing and running applications in containerized environments. Containers allow a user to deploy an application and all its dependencies so that the application always runs identically no matter what machine it's on. All six of the components of DNS Guard were Dockerized (the frontend, the backend, the AI service, PostgreSQL, Redis and Qdrant) using Docker. These Docker images were then set up and connected to one another using Docker Compose in a single configuration file. This allowed us to bring up the entire system using a single command, greatly simplifying the deployment process and user setup compared to what would be necessary without Docker (e.g. Requiring manual installation and configuration of python, node.js, PostgreSQL, Redis, Qdrant, etc).

Design: Analysis, Design Methodology and Implementation Strategy

3.1 Function of System

DNS Guard is a web-based domain threat intelligence platform. As such, the main functionality is to take a domain name as input from the user and to analyze it on multiple dimensions: DNS records are retrieved and verified (DNSSEC verification), WHOIS is queried, certificate transparency log is consulted, passive DNS history is queried, a search is performed for typosquatting domain names, the subdomains of the given domain are enumerated, a reverse IP address lookup is performed, threat intelligence blocklists are consulted, and the system finally gives a machine-learning-based threat score to the given domain name. All information are displayed on a web interface and can also be exported as PDF report. A specific part of the website, an AI chatbot, answers to questions about the result as well as general questions related to DNS security.

This system has been developed as a set of microservices. The front-end communicates with the back-end via a set of REST API endpoints. Analysis which take a long time are delegated to some Celery workers. The AI service performs separately threat scoring and handles the AI chatbot queries. All services have in common a PostgreSQL database and communicate via Redis.

3.1.1 Use Case Diagram

This use case diagram illustrates all possible interactions between a user and DNS Guard as well as the inner workings that occur as a result of those interactions. There are two different actor types in this diagram: The **User** which is the actual person interacting with the web interface, and the **External Services** which are the third party services DNS Guard communicates with when performing an analysis; some of which include DNS Servers, WHOIS servers, Certificate Transparency logs and blocklist providers.

The user may perform up to five actions. Their primary action, which causes eight subprocesses to execute within the system: checking of DNS records, WHOIS lookup, checking of certificate transparency logs, threat intelligence checking, performing ML threat scoring, detection of typosquatting, sub-domain enumeration and reverse IP lookup, is to submit a do-

main for analysis. All eight sub-processes are executed within the same analysis job. The other actions a user may take are to view the results on the analysis dashboard, to download a PDF report, to use the AI chatbot to pose questions, and to review their history of recent searches, which is stored locally in the browser without the need for server interaction.

This use case diagram serves an important role in defining the system boundary.

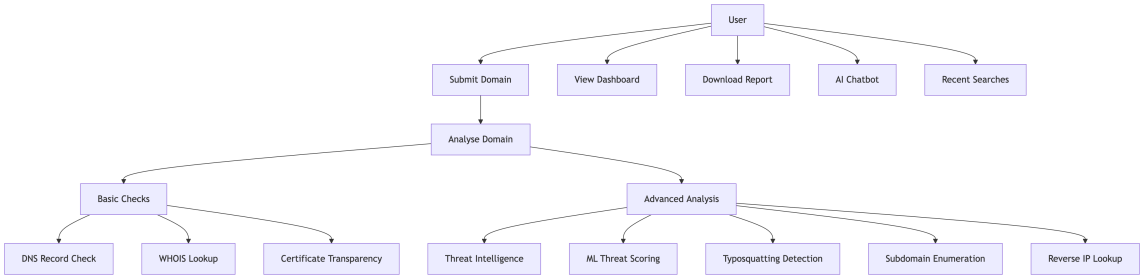

Figure 3.1: Use Case Diagram for DNS Guard

3.1.2 Activity Diagram

The activity diagram displays the sequential workflow of activities that happen from the user entering a domain, through to showing the results on the screen. This covers the actions the user sees and the backend processes they don't.

The process is triggered when the user enters a domain into the search box, then clicks the analyse button. The input is validated on the frontend, then a POST request with the domain name is made to the backend API. The backend accepts the POST, then create a new Celery task. The ID for the task is immediately returned to the frontend. The reason for doing this is that it prevents the API from waiting while the task runs. The frontend, using the returned ID, then poll the API every few seconds to see when it is complete.

The analysis modules are executed in a Celery worker pick-up the task from the Redis queue, run each analysis module (in parallel where appropriate to speed them up), DNS look-up, WHOIS look-up, certificate transparency, passive DNS history, typo-squatting analysis, subdomain enumeration, reverse IP lookup and threat intelligence look-ups are all performed. Simultaneously, the feature extractor forms a 40-feature vector out of the DNS data and provides it to the AI service which returns a threat score and feature importance measures.

After all modules have completed, the results are formatted into a result object, stored in PostgreSQL, and marked as complete in Redis. This data will then be retrieved on the next poll by the frontend, which renders the analysis dashboard showing threat score,DNS results, WHOIS data, certificate information, map of geo location, and everything else.

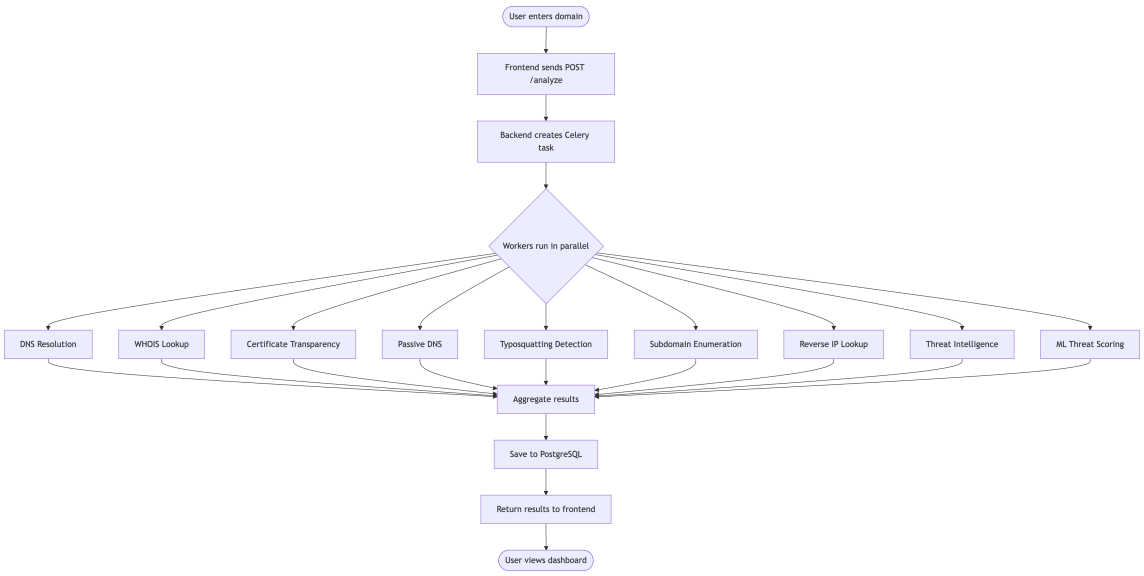

Figure 3.2: Activity Diagram for Domain Analysis

3.1.3 Sequence Diagram

The sequence diagram depicts the precise order that the various components of DNS Guard receive messages during a domain analysis. Unlike the activity diagram, the sequence diagram focuses on not just the flow of actions but on the components receiving messages from one another.

The diagram has seven participants: the User's browser, Next.js, FastAPI, Celery, the AI service, PostgreSQL, and Redis.

The sequence begins when the user sends a domain for analysis. The user's browser submits a POST request to the backend's `/analyze` endpoint. The backend simply takes in the user request, pushes a task onto the Redis queue, and immediately returns a task ID to the user. This ensures the API doesn't lag and HTTP connections don't timeout during a time-consuming domain analysis.

From here, the frontend begins a polling loop, hitting an endpoint on the API (specifically `/analyze/status/{task_id}`) periodically (every few seconds) to check on the task's progress. In the background, Celery picks up the task, begins analysis, and sends POST requests to the AI service's `/score` endpoint with the extracted feature vector to retrieve a threat score. The AI service then uses an ensemble of models to determine a threat score and return a set of feature importance values. Celery saves the results to PostgreSQL and then marks the task as complete in Redis.

On the next poll by the frontend, the backend sees that the task has been marked as complete and retrieves the results from PostgreSQL, then sends the analysis data to the frontend, which finally renders a full dashboard.

The system, therefore, is non-blocking and can handle multiple requests simultaneously as

analyses occur concurrently via Celery tasks.

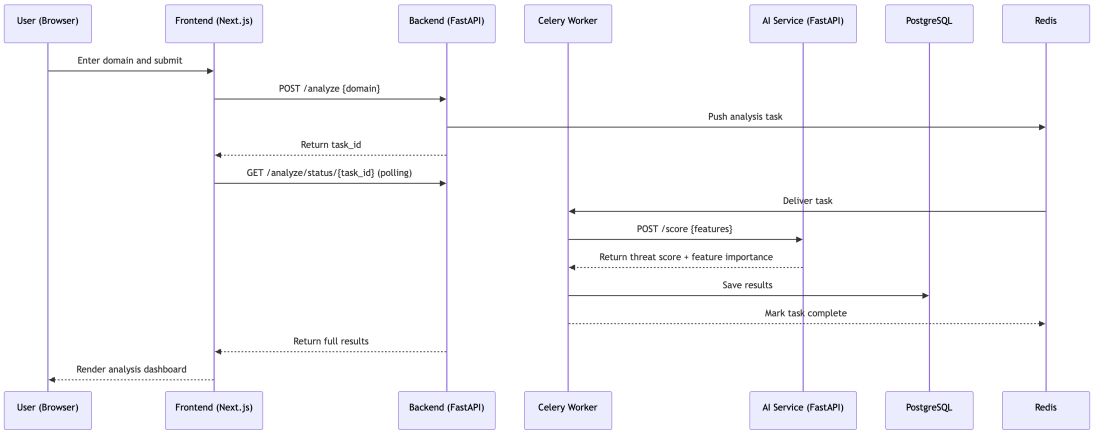

Figure 3.3: Sequence Diagram for Domain Analysis Request

3.2 Data Modelling

3.2.1 Entity-Relationship Diagram

The following figure is an ER diagram of the PostgreSQL database structure used by DNS Guard. It describes the core entities, their fields, and the relationships among entities.

The core entity is the **Scan**, each one representing one pass of analysis on a given domain. A new entry in this table is made each time the user requests a scan on a domain. It records the creation date and status of the scan. Each Scan belongs to a **Domain** entity storing the domain name, along with the date the domain was first seen by the system, and date it was last seen by the system, a Domain can have multiple Scans during its life cycle enabling the system to compare between different scans over time. Each Scan entity is associated with many **DNS_Record** entries, one for each DNS record obtained while running the scan (A records, MX records, NS records, TXT records, and various DNSSEC related records). Having them as separated entries instead of a big JSON blob makes it easier to query and compare them. The Scan entity has exactly one **WHOIS_Record**, storing information such as the domain registrar name, the domain creation and expiry dates, and the raw response of the WHOIS query. A new entity **WHOIS_Diff** is related to a Domain and stores the changes detected between 2 consecutive WHOIS scans. The system can thus notify the user about possible domain hijacking based on those changes. Lastly, each Scan entity is associated with a **Threat_Score** which holds the ML output as a number, and feature importances as a JSON blob.

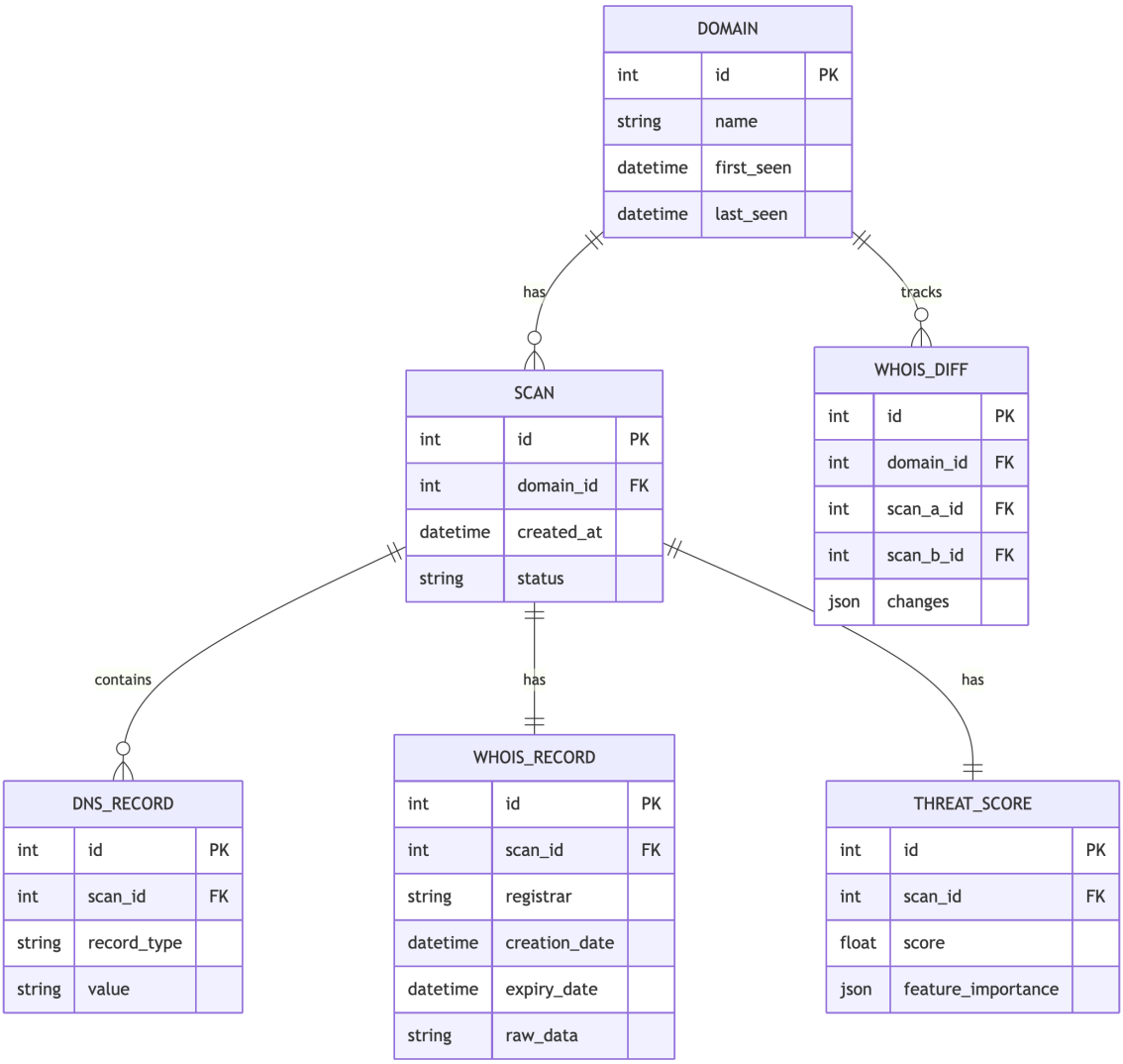

Figure 3.4: Entity-Relationship Diagram for DNS Guard Database

3.2.2 Class Diagram

The class diagram illustrates the main classes in the DNS Guard backend and their relationship. It clearly displays the structure of the code and how responsibility is shared among different components of the system.

The main class is **AnalysisService**, it acts as the orchestrator for the whole analysis process. After a Celery worker takes over an analysis request, it will invoke `run()` on the `AnalysisService` class, passing in the domain name. This method is responsible to invoke all individual service classes one by one and collect results.

Each service class takes charge of a single responsibility. **DNSResolver** looks upDNS servers and returns all DNS records about the domain. **WHOISLookup** querysWHOIS servers and returns ownership information of the domain. **CertTransparency** querys certificate transparency logs from crt.sh. **PassiveDNS** querys history DNS resolutions information about the domain. **TyposquatDetector** compares the domain against the list of popular domains by ap-

plying edit-distance. **SubdomainEnum** discovers new subdomains by both using a bruteforce wordlist and from certificate transparency data. **ReverseIP** finds other domains resolving to the same IP. **ThreatIntel** checks the domain against Spamhaus DBL and PhishTank. **ThreatScorer** pass the obtained feature vector to the AI service, and returns the threat score and the features that are most important to this threat score.

Each service class returns a typed result object. All of these result objects are combined into one **AnalysisResult** object by the `AnalysisService`. Then this object will be serialized and stored in the database. This design allows easy testing and extending of the codebase. Adding a new analysis requires just developing a new service class and adding the invocation for this service into `AnalysisService`.

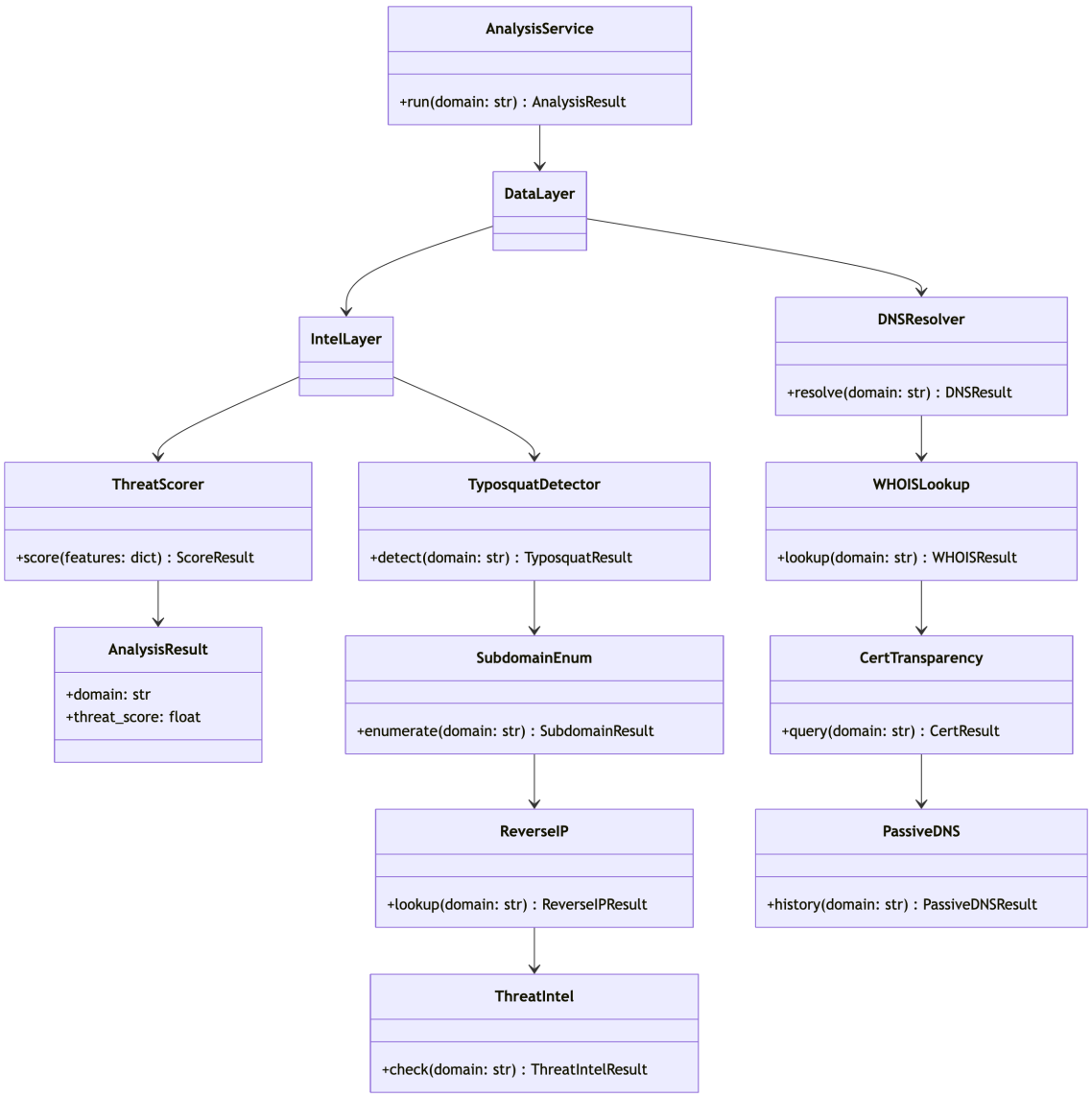

Figure 3.5: Class Diagram for DNS Guard Backend

3.3 Functional and Behavioural Modelling

3.3.1 Data Flow Diagram

The Data Flow Diagram (DFD) presents the input of the data into, through and out of the system (DNS Guard). Two levels of DFD are provided. The Level 0 of the DFD show a overview of the system as a single process and the Level 1 of the DFD break the single process into its sub processes.

Level 0 (Context Diagram): At this level the system (DNS Guard) is presented as a black box where an input of the domain name can be observed while the output is an analysis report and also connections to four data bases that can be observed in the data flow of data from and to. Data flows in both directions between the system (DNS Guard) and the DNS server (resolves DNS records), WHOIS server (retrieves information about the domain), crt.sh certificate transparency log (retrieves information about SSL/TLS certificates), and Spamhaus and PhishTank blocklist services (checks if the domain is blacklisted by the two services).

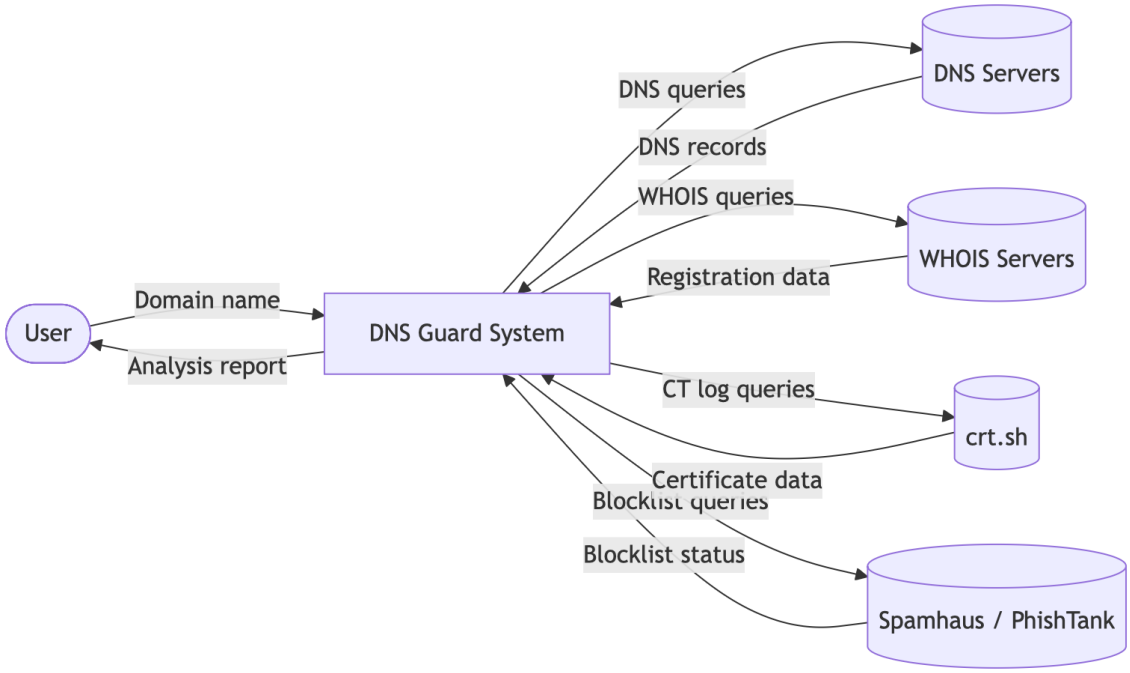

Figure 3.6: Data Flow Diagram – Level 0 (Context Diagram)

Level 1 (DFD detailed): At this level, the internal workings of the DNS Guard system are decomposed into 6 numbered processes. Process 1 (Data collection) takes the users domain and queries all external services for raw data. Process 2 (Feature extraction) takes the raw DNS and WHOIS data and translates it to the 40-feature vector used by the ML model. Process 3 (ML Threat scoring) takes the feature vector, and outputs the threat score and feature weights. Process 4 (Threat intelligence check) runs in parallel with the feature extraction and compares

the domain to the relevant blocklists. Process 5 (Result aggregation) amalgamates all previous processes' results into one object which is stored in the PostgreSQL database. Process 6 (Report generation) takes the aggregated result and, when requested, returns it as a PDF file. The user can take the dashboard information and the PDF file from the system.

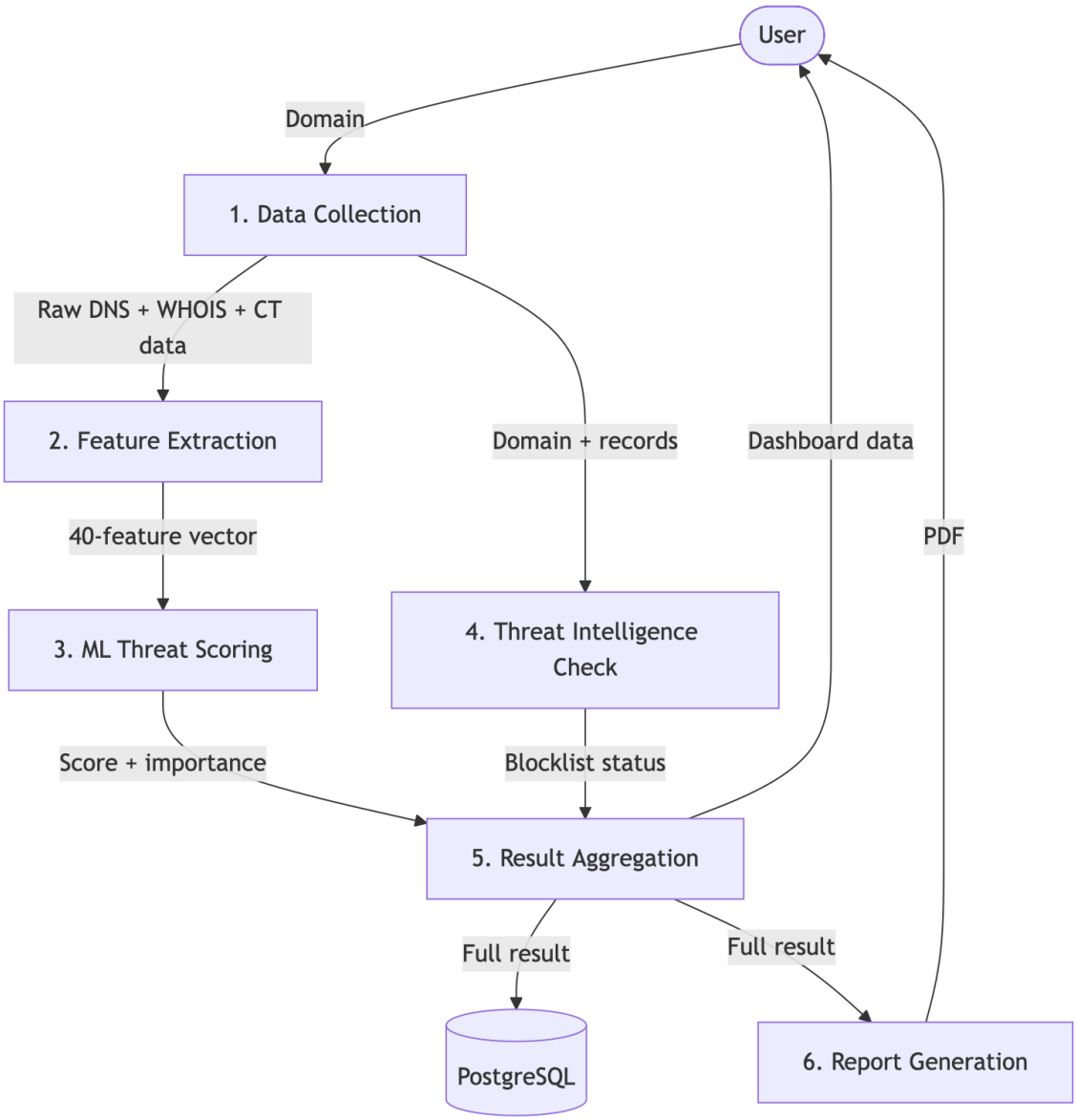

Figure 3.7: Data Flow Diagram – Level 1

3.3.2 Data Dictionary

The data dictionary below defines the key data elements used in DNS Guard. It lists the field name, data type, and a description of what each field stores and how it is used in the system.

Table 3.2: Data Dictionary

Field Name	Data Type	Description
`domain`	String	The domain name submitted by the user for analysis (e.g. `example.com`).
`scan_id`	Integer	A unique identifier assigned to each analysis scan. Used to retrieve results and track task status.
`created_at`	Datetime	The date and time when the scan was created. Stored in UTC.
`status`	String	The current status of the scan task. Possible values are `pending`, `running`, and `complete`.
`threat_score`	Float	A numerical risk score between 0 and 100 produced by the ML ensemble model. A score above 70 is considered high risk.
`feature_importance`	JSON	A dictionary mapping each of the 40 input features to its contribution to the threat score. Used to generate the SHAP-style bar chart.
`dns_records`	JSON	A list of DNS records retrieved for the domain. Each entry includes the record type (A, MX, NS, TXT, etc.), name, and value.
`dnssec_valid`	Boolean	Indicates whether the domain has a valid and complete DNSSEC configuration, including DNSKEY, RRSIG, and DS records.
`registrar`	String	The name of the domain registrar as returned by the WHOIS lookup (e.g. GoDaddy, Namecheap).
`creation_date`	Datetime	The date the domain was first registered, taken from WHOIS data. Used to flag newly registered domains.
`expiry_date`	Datetime	The date the domain registration is due to expire, taken from WHOIS data. Used to flag soon-to-expire domains.
`whois_diff`	JSON	A structured record of changes detected between two consecutive WHOIS scans of the same domain. Includes the field name, old value, and new value for each change.

Field Name	Data Type	Description
`cert_count`	Integer	The total number of SSL/TLS certificates found in certificate transparency logs for the domain. A sudden spike may indicate suspicious activity.
`has_wildcard_cert`	Boolean	Indicates whether any of the certificates found are wildcard certificates (e.g. `*.example.com`).
`passive_dns_ips`	JSON	A list of historical IP addresses that the domain has resolved to, retrieved from passive DNS data sources.
`fast_flux_detected`	Boolean	Indicates whether fast-flux behaviour was detected. Fast-flux is identified by a large number of different IP addresses in the passive DNS history over a short period.
`typosquat_matches`	JSON	A list of known brand domains that are similar to the submitted domain, along with their edit-distance similarity scores.
`subdomains`	JSON	A list of subdomains discovered through brute-force wordlist enumeration and certificate transparency log queries.
`reverse_ip_domains`	JSON	A list of other domain names found to be hosted on the same IP address as the submitted domain. A very large number of co-hosted domains may indicate shared malicious hosting.
`on_spamhaus_dbl`	Boolean	Indicates whether the domain appears on the Spamhaus Domain Block List, which covers spam, phishing, and malware domains.
`on_phishtank`	Boolean	Indicates whether the domain appears in the PhishTank database of verified phishing URLs.
`geo_locations`	JSON	A list of geolocation records for each IP address associated with the domain. Each entry includes country, city, latitude, and longitude.

Field Name	Data Type	Description
`spf_record`	String	The SPF TXT record found for the domain, if present. An absent or incorrectly configured SPF record leaves the domain open to email spoofing.
`dmarc_record`	String	The DMARC TXT record found for the domain, if present. DMARC builds on SPF and DKIM to provide a policy for handling unauthenticated email.
`dkim_record`	String	The DKIM TXT record found for the domain, if present. DKIM allows receiving mail servers to verify that an email was sent by an authorised server.

Implementation

4.1 Implementation Environment

4.1.1 Model Used in Developing

DNS Guard has been developed with the **Incremental Development Model**. The system is developed and delivered in small pieces. With each increment of the system, new functions will be added upon that system and tested. The model was adopted because there are a large number of separate modules in the project, which includes DNS resolution module, WHOIS lookup module, certificate transparency module, and machine learning scoring module. The system can be developed one by one and integrated.

The development went through the following increments:

1. **Increment 1 – Core Infrastructure:** Set up the Docker Compose environment with FastAPI backend, Next.js frontend, PostgreSQL, and Redis. Implemented the basic domain submission and task queuing flow using Celery.

2. **Increment 2 – DNS and WHOIS Analysis:** Implemented DNS record resolution using dnspython, DNSSEC validation, and WHOIS lookup. Added the results display on the frontend.

3. **Increment 3 – Threat Intelligence Modules:** Added certificate transparency querying, passive DNS history, typosquatting detection, subdomain enumeration, reverse IP lookup, and blocklist checks.

4. **Increment 4 – Machine Learning:** Built the feature extractor, trained the Random Forest and MLP ensemble model, and integrated the AI service with the backend.

5. **Increment 5 – AI Chatbot:** Set up the Qdrant vector database, built the RAG pipeline, and integrated the Groq API (LLaMA 3.1) with a rule-based fallback.

6. **Increment 6 – Reporting and Polish:** Added PDF report generation, the geolocation map, WHOIS diff detection, dark/light mode, and recent search history.

This incremental approach meant that a working version of the system was available at the end of each increment, which made it easy to test and get feedback throughout the development process.

4.1.2 Software Prototyping Types

While developing DNS Guard, an **evolutionary prototyping** process was applied to the incremental model. In evolutionary prototyping, the prototype developed does not discarded after the user feedback but continuously developed into the final system. This approach suited the project as few features such as threat scoring model and chatbot had uncertain requirements which developed as the project moved along.

For instance, initially, lexical features were used in the threat scoring model. Upon testing, it was observed that DNS record features and DNSSEC status were also relevant so the features were increased from 12 to 40 features. Likewise, initially the chatbot used simple keyword matching rule-based engine which later evolved into using Groq API (LLaMA 3.1 8B Instant) for Natural Language response.

4.2 Coding Standard

Consistent coding standards were followed throughout the project to keep the codebase readable and maintainable.

Python (Backend and AI Service):

- Code follows the **PEP 8** style guide [30], which defines conventions for indentation, naming, and line length.
- All functions and classes have docstrings explaining their purpose, parameters, and return values.
- Variable and function names use `snake_case`. Class names use `PascalCase`.
- Type hints are used throughout, as required by FastAPI for request and response validation.
- Each service module is kept in its own file under the `services/` directory, with a single responsibility.

TypeScript (Frontend):

- Code follows the **Airbnb TypeScript style guide** conventions.
- All React components are written as functional components with typed props.
- Component names use `PascalCase`. Variables and functions use `camelCase`.

- Each component is kept in its own file under the `components/` directory.
- API calls are made through Next.js API route proxies rather than directly from the browser, to avoid exposing backend URLs.

General:

- All sensitive configuration values such as database passwords and API keys are stored in environment variables and never hardcoded in source files.
- Git was used for version control throughout the project.

4.3 Laboratory Setup

DNS Guard was developed and tested on the following hardware and software environment:

Table 4.3: Laboratory Setup

Component	Details
Operating System	macOS
Processor	Apple Silicon (ARM64)
RAM	18 GB
Storage	1 TB SSD
Docker Version	Docker Desktop 4.x
Python Version	3.11
Node.js Version	20.x (LTS)
Browser for Testing	Google Chrome (latest)
Code Editor	Visual Studio Code
Version Control	Git with GitHub
LLM (Optional)	Groq API (LLaMA 3.1 8B Instant)

All the 6 services (frontend, backend, ai service, postgresql, redis and Qdrant) were running on a local setup with docker-compose. The system was then tested analyzing various real world as well as artificial domains; these include known malicious domains obtained from public threat feeds and valid domains from Alexa top 1000 list.

4.4 Tools and Technology Used

The tools and technologies used in the implementation of DNS Guard are described in detail in Section 2.4. A brief summary is provided here for reference:

Table 4.4: Tools and Technologies Used

Tool / Technology	Version	Role in Project
Next.js	14	Frontend web application
Tailwind CSS	3.x	Frontend styling
FastAPI	0.110+	Backend API and AI service
Celery	5.x	Asynchronous task queue
PostgreSQL	15	Persistent data storage
Redis	7.x	Message broker for Celery
Qdrant	1.x	Vector database for RAG
scikit-learn	1.4+	ML model training and inference
dnspython	2.x	DNS record resolution
ReportLab	4.x	PDF report generation
Groq API	Cloud	Fast LLM inference (free tier)
Docker Compose	2.x	Service orchestration

4.5 Screenshots / Snapshots

This section presents screenshots of the DNS Guard web application showing the key features of the system.

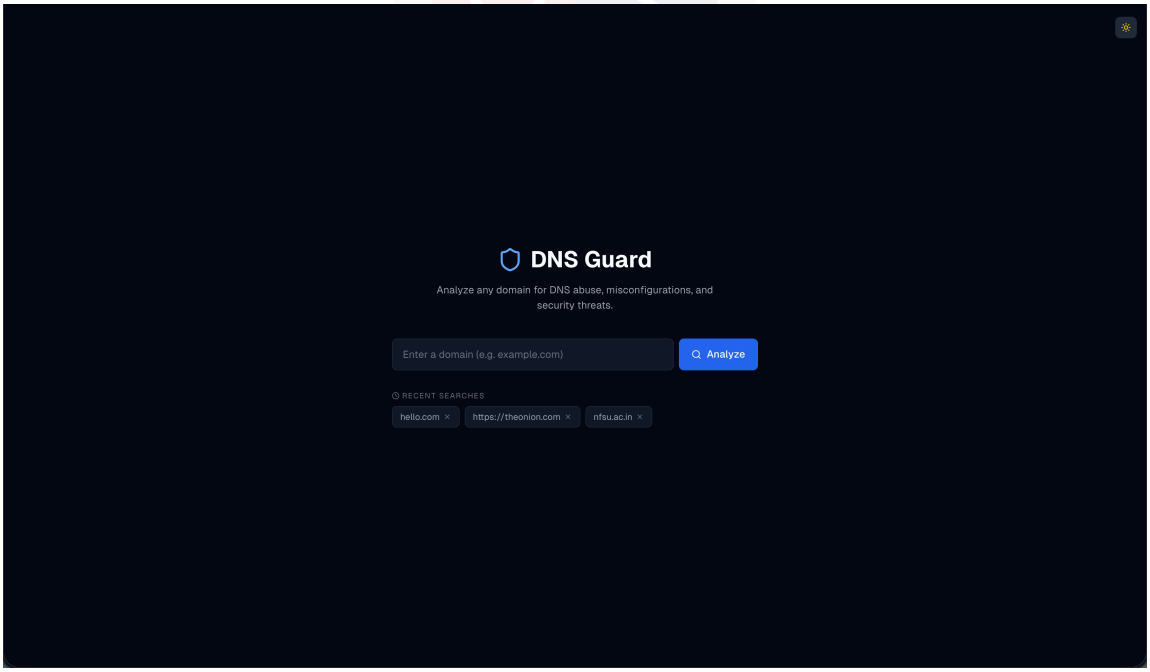

Figure 4.8: DNS Guard – Home Page with Search Interface

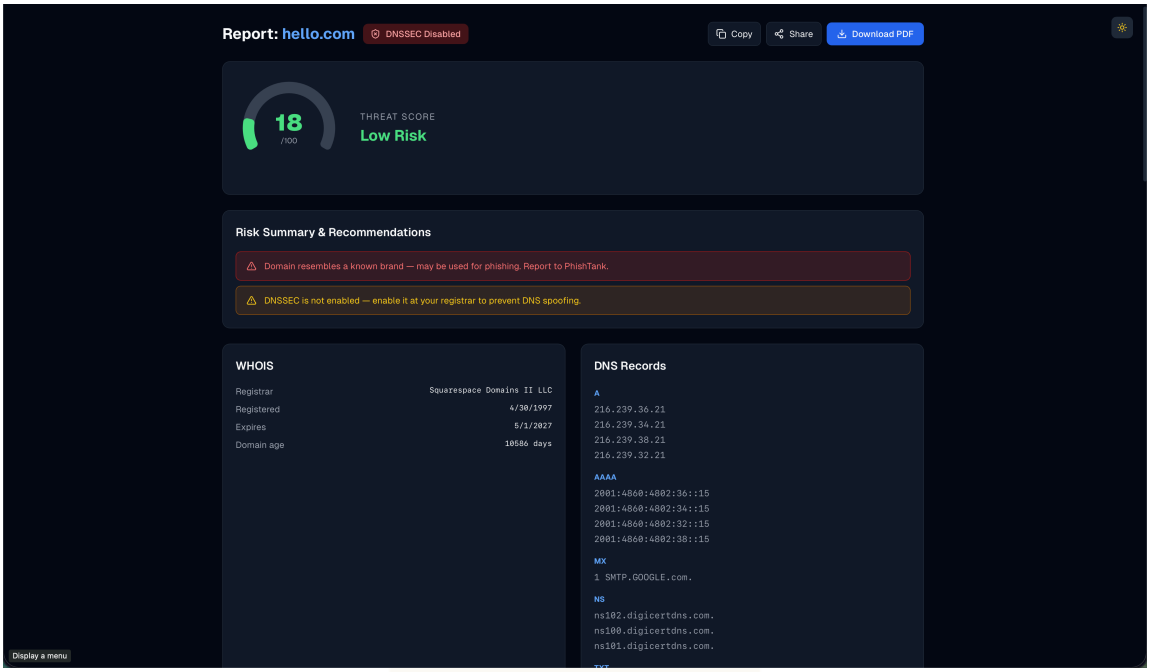

Figure 4.9: DNS Guard – Analysis Dashboard (Part 1: Threat Score and DNS Records)

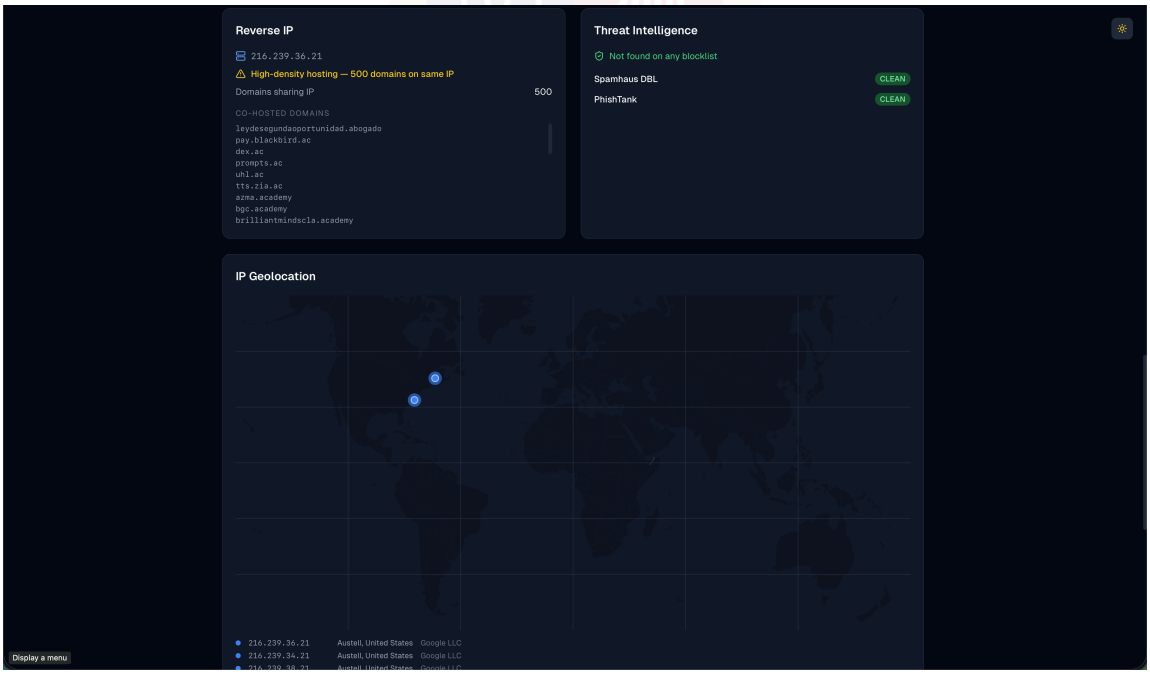

Figure 4.10: DNS Guard – Analysis Dashboard (Part 2: WHOIS and Certificate Transparency)

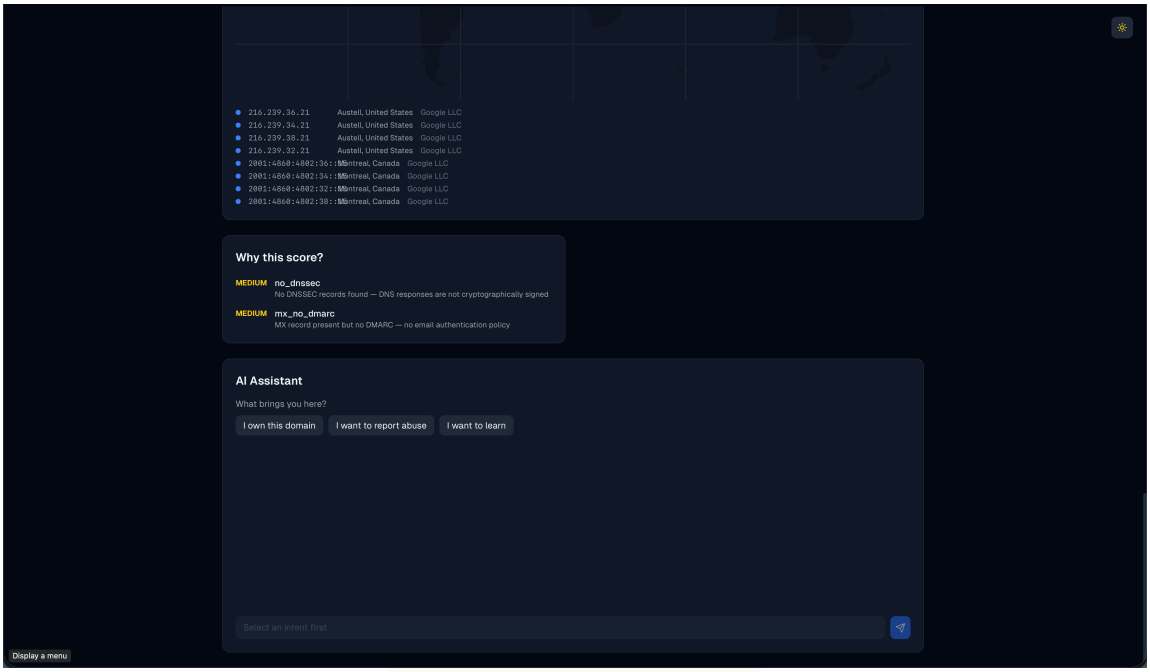

Figure 4.11: DNS Guard – Analysis Dashboard (Part 3: Threat Intelligence and Geolocation)

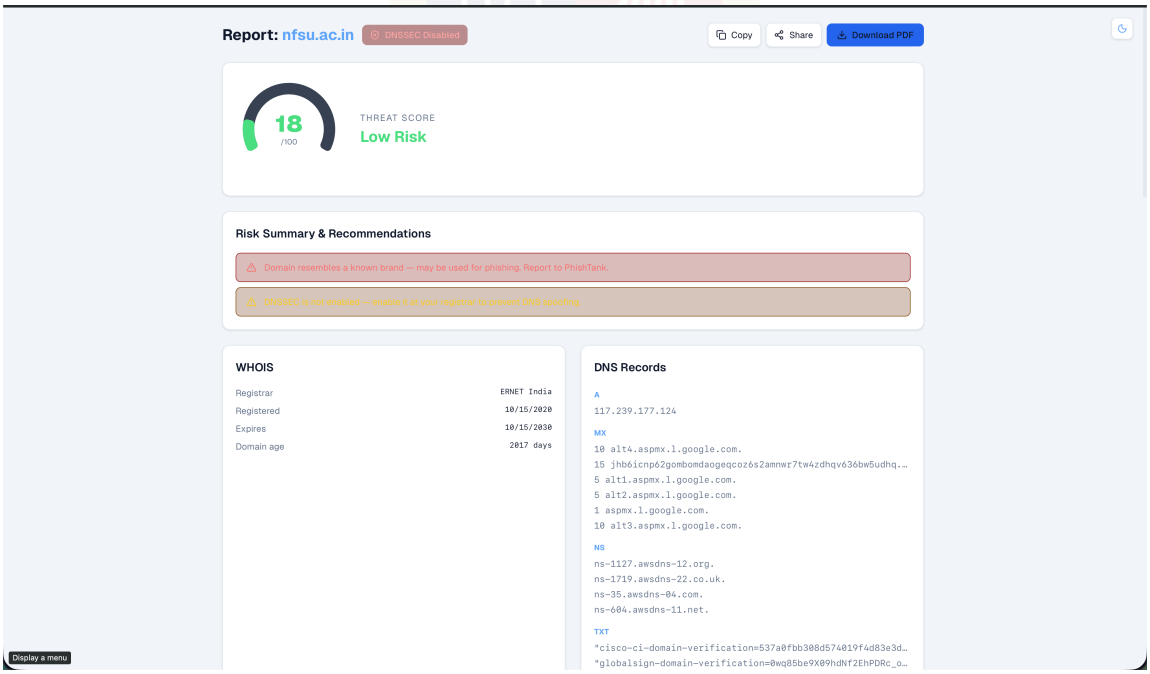

Figure 4.12: DNS Guard – Light Mode Interface

Summary of Results and Future Scope

5.1 Advantages and Unique Features

Some advantages of DNS Guard compared to existing tools and several unique features that are typically not found in the same free solution:

All-in-one analysis: Most current tools target one part of DNS security. DNS Guard aims to combine the analysis of DNS records, DNSSEC validation, WHOIS look-up and change detection, certificate transparency monitoring, passive DNS look-up, typosquatting look-up, subdomain enumeration, reverse IP lookup, threat intelligence look-up into one scan. No need to open multiple tabs and correlate the data manually.

Machine learning threat score with explainability: A threat score between 0 and 100 is assigned to each domain based on the output of a combined Random Forest classifier and MLP neural network, trained on the 40 features calculated in previous stages. Accompanying the score is a chart showing feature importance to understand what influenced the threat score. This avoids black-box number reporting.

WHOIS change detection: The WHOIS data of each scanned domain is stored and compared to previous results from another scan of the same domain. If the WHOIS information changes (registrar, name servers, registrant info), this is shown in the results page, providing a method for detect domain hijacking or unauthorized transfers.

AI chatbot with Groq: A built-in AI chatbot can provide answers to questions about DNS security and threat intelligence, using the Groq API (LLaMA 3.1 8B Instant). A prompt containing a system description relevant to the current domain scan and the user's question is passed to the Groq API. A rule-based fall-back is present to ensure some chatbot function even if the Groq API isn't available.

Free tier API: The chatbot utilises the free tier of the Groq API to provide free, fast LLM responses. No API key is required from the user, if not provided the rule-based system is used instead which covers the most important DNS security aspects.

Free & Open deployment: The entire system can be deployed via Docker Compose in a single command and works without paid API keys, making it suitable for students, researchers and small organisations.

PDF Report Generation: Each scan can be exported as a comprehensive PDF report, detailing all detected information, the threat score, feature importance and suggested remedia-

tions, to ease reporting.

Dark and Light mode: The interface supports both dark and light mode via a toggle to suit different user preferences.

5.2 Results and Discussions

The system was evaluated by analyzing domains from several categories across different risk levels. These categories included known malicious domains from public threat feeds, recently registered domains, domains missing email security records, legitimate high-traffic domains, and typosquat domains.

Threat scoring accuracy: High threat scores (¿70) were given to domains found on blocklists or having multiple risk factors such as high lexical entropy, absent DNSSEC, and missing SPF and DMARC records. Domains within Alexa's top 1000 were consistently low scoring (¡30). The model was strong in correctly identifying domains with obvious risks and demonstrated reasonable uncertainty in ambiguous cases, as expected for the problem.

DNSSEC validation: Domains with correct DNSSEC configuration and lack thereof with missing/invalid DNSKEY, RRSIG, or DS records were accurately identified. Most tested domains did not have DNSSEC enabled, aligning with global reports on limited DNSSEC adoption [31].

Typosquatting detection: The edit-distance based typo detection mechanism accurately identified domains with a 1 or 2 character difference from known brands, such as the identification of `g00gle.com` and `paypa1.com`.

Certificate transparency: All domains returned valid data via crt.sh, with domains having numerous recent or wildcard certificate issuances being flagged.

Passive DNS and fast-flux detection: Domains with available passive DNS data identified fast-flux behavior correctly through high IP address resolution count over a short period.

Chatbot performance: When using Groq, the chatbot was precise and contextually relevant to DNS security questions with sub-second responses owing to the LPU inference. The fallback handled common queries when the API key was missing.

Performance: Each domain analysis averaged 8-15 seconds depending on external service response times (WHOIS, crt.sh). The asynchronous Celery system ensured the API was responsive, and analysis of multiple domains was parallel.

5.3 Future Scope of Work

Although DNS Guard is a usable and well rounded platform there are various areas it can be expanded or enhanced in future work.

Real-time monitoring and alerting: At present, the only analysis done by DNS Guard is when a user requests the threat score for a domain. A possible future enhancement would

involve the addition of a monitoring system. A list of 'watched' domains can be maintained and the system will periodically rescan these domains and alert the user when there has been a drastic change in threat score or a modification in WHOIS data. This will be beneficial for organisations wanting to monitor their own domain names and even competitor's domain names.

Expanded threat intelligence feeds: At present, only Spamhaus DBL and PhishTank is used. Future versions of the application could incorporate other intelligence feeds such as the SURBL multi-blocklist [32], OpenPhish [33] and abuse.ch URLhaus [34]. Using a broader range of intelligence feeds would improve detection accuracy.

VirusTotal and Shodan integration: A facility already exists for the optional inclusion of API keys for VirusTotal and Shodan. A more in depth use could be made of these APIs such as including VirusTotal's scan history of a domain and information retrieved using Shodan regarding the services running on a domain's IP addresses.

Improved machine learning model: At present a 40-feature vector with a Random Forest and MLP ensemble is used. Further research could include using deep learning methods for classifying domains such as character level convolutional neural networks or transformer networks which have already achieved some success in DGA classification research [16]. A larger training dataset could be gathered with more malicious samples.

User accounts and scan history: Results are stored in a database but no user accounts exist. An account based system would enable users to store and access their past scans, set up lists of domains to monitor and receive automated alerts based on conditions they specify.

API access: An interface using RESTful API would provide external applications with the ability to check a domain's threat score programmatically. Such an API could be a public, rate limited and authenticated interface providing a beneficial feature as part of a broader security infrastructure.

Automated remediation guidance: Recommendations are given based on the calculated risk level. These recommendations can be further expanded into producing automatically generated configuration statements or other guidance which the user can then paste into their DNS provider.

Internationalized domain name support: As the use of homograph attacks increases through IDNs, an enhancement for detecting these types of attacks could be included [3].

Mobile friendly interface: While the current interface is responsive it was designed with desktops in mind, providing a mobile friendly design would increase accessibility on phones and tablets.

Conclusion

The initial aim of this project was to create a feature-rich, accessible, free tool to perform domain security analysis, and the resulting system, DNS Guard, a web-based domain threat intelligence platform combines various areas of DNS security analysis into one platform.

This system was successful at achieving every objective of the project from the initial goals at the beginning of the year. The system can analyse any given domain name across ten dimensions; including DNS record analysis, DNSSEC validation, WHOIS look-up with change detection, Certificate Transparency monitoring, Passive DNS history with fast-flux detection, typosquatting detection, subdomain enumeration, Reverse IP lookup and threat intelligence blocklist checks. All of these findings are then provided in a clean and comprehensible dashboard suitable for both technical and non-technical users.

The machine learning aspect of the system achieved a working threat scoring model which uses a 40 feature vector which uses an ensemble of a Random Forest classifier with an MLP neural network to return a score between 0 and 100 that indicates the risk level of a domain and features that are most prominent in returning this score are displayed via a SHAP-style feature importance chart. This allows for an important aspect to security which is the explainability; where the user can understand why a domain has been flagged and take appropriate action.

The AI chatbot system that was developed utilizes the Groq API with an LLaMA 3.1 LLM, backed up by a rule-based fallback mechanism; which gives the user the ability to question the AI in natural language regarding DNS security or even question it with regards to a scan's findings. Because the LLM runs locally, none of the user's data is passed to an external provider which is a vital security consideration.

The system architecture relies upon a group of containerised microservices running with the use of Docker Compose which has been proven to ease development, testing and deployment of the system. With the use of FastAPI, Celery, PostgreSQL and Redis a solid, scalable and extensible system has been developed which could be scaled up for a number of users with relative ease.

On a more general note, this project proves that it is achievable to create a feature-rich and practical DNS security tool using 100% free and open-source technologies. The current fragmented and time consuming nature of assessing the security posture of a domain, usually requiring a number of disparate tools over many web-pages, is solved directly by DNS Guard.

Finally, this project shows just how far reaching the problems with DNS security can reach;

the number of domains which are fundamentally misconfigured (e.g. Missing SPF/DKIM/DMARC, no DNSSEC or new and suspiciously similar looking domains for existing popular brands) is incredible. These issues aren't too difficult to resolve, but first, the issue of awareness must be met.

Overall, DNS Guard is a fully functional, well-structured and extremely useful final year project that covers numerous aspects including networking, cybersecurity, machine learning, Natural Language Processing and full-stack web-development. The work done in this project provides a fantastic base for further development and could become a real-world security tool with enough further hardening.

References

[1] P. Mockapetris, "Domain names – concepts and facilities," Internet Engineering Task Force, RFC 1034, 1987. [Online]. Available: https://www.rfc-editor.org/rfc/rfc1034

[2] S. Banerjee, S. Bhatt, B. Krishnamurthy, and M. Mah, "Detecting typosquatting domains," in *Proceedings of the 2008 ACM Workshop on Recurring Malcode*, ACM, 2008, pp. 1–8.

[3] T. Holz, C. Gorecki, K. Rieck, and F. C. Freiling, "Measuring and detecting fast-flux service networks," in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, Internet Society, 2008.

[4] S. Kitterman, "Sender policy framework (spf) for authorizing use of domains in email," Internet Engineering Task Force, RFC 7208, 2014. [Online]. Available: https://www.rfc-editor.org/rfc/rfc7208

[5] R. Arends, R. Austein, M. Larson, D. Massey, and S. Rose, "Dns security introduction and requirements," Internet Engineering Task Force, RFC 4033, 2005. [Online]. Available: https://www.rfc-editor.org/rfc/rfc4033

[6] Anti-Phishing Working Group, "Phishing activity trends report, q4 2023," APWG, Tech. Rep., 2023. [Online]. Available: https://apwg.org/trendsreports/

[7] VirusTotal, *Virustotal – free online virus, malware and url scanner*, Accessed: April 2026, 2024. [Online]. Available: https://www.virustotal.com

[8] MXToolbox, *Mxtoolbox – email header analyzer, mx record lookup*, Accessed: April 2026, 2024. [Online]. Available: https://mxtoolbox.com

[9] Shodan, *Shodan – the search engine for the internet of everything*, Accessed: April 2026, 2024. [Online]. Available: https://www.shodan.io

[10] SecurityTrails, *Securitytrails – dns intelligence and historical data*, Accessed: April 2026, 2024. [Online]. Available: https://securitytrails.com

[11] HackerTarget, *Dnsdumpster – dns recon and research*, Accessed: April 2026, 2024. [Online]. Available: https://dnsdumpster.com

[12] PhishTank, *Phishtank – free community site for phishing data*, Accessed: April 2026, 2024. [Online]. Available: https://www.phishtank.com

[13] Spamhaus, *The spamhaus project – domain block list*, Accessed: April 2026, 2024. [Online]. Available: https://www.spamhaus.org

[14] M. Antonakakis, R. Perdisci, W. Lee, N. Vasiloglou, and D. Dagon, "Detecting malware domains at the upper dns hierarchy," in *Proceedings of the 20th USENIX Security Symposium*, USENIX Association, 2011.

[15] L. Bilge, E. Kirda, C. Kruegel, and M. Balduzzi, "Exposure: Finding malicious domains using passive dns analysis," in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, Internet Society, 2011.

[16] J. Woodbridge, H. S. Anderson, A. Ahuja, and D. Grant, "Predicting domain generation algorithms with long short-term memory networks," in *arXiv preprint arXiv:1611.00791*, 2016.

[17] Z. C. Lipton, "The mythos of model interpretability," *Queue*, vol. 16, no. 3, pp. 31–57, 2018. DOI: 10.1145/3236386.3241340

[18] B. Halley and B. Westenberg, *Dnspython – a dns toolkit for python*, Accessed: April 2026, 2024. [Online]. Available: https://www.dnspython.org

[19] F. Pedregosa et al., "Scikit-learn: Machine learning in python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[20] S. Ramirez, *Fastapi – modern, fast web framework for building apis with python*, Accessed: April 2026, 2024. [Online]. Available: https://fastapi.tiangolo.com

[21] Vercel, *Next.js – the react framework for the web*, Accessed: April 2026, 2024. [Online]. Available: https://nextjs.org

[22] Docker Inc., *Docker – accelerated container application development*, Accessed: April 2026, 2024. [Online]. Available: https://www.docker.com

[23] The PostgreSQL Global Development Group, *Postgresql – the world's most advanced open source relational database*, Accessed: April 2026, 2024. [Online]. Available: https://www.postgresql.org

[24] Redis Ltd., *Redis – the real-time data platform*, Accessed: April 2026, 2024. [Online]. Available: https://redis.io

[25] Qdrant, *Qdrant – vector database for the next generation of ai applications*, Accessed: April 2026, 2024. [Online]. Available: https://qdrant.tech

[26] Groq Inc., *Groq api – fast ai inference with lpu technology*, Accessed: April 2026, 2024. [Online]. Available: https://console.groq.com

[27] Tailwind Labs, *Tailwind css – a utility-first css framework*, Accessed: April 2026, 2024. [Online]. Available: https://tailwindcss.com

[28] A. Solem and contributors, *Celery – distributed task queue*, Accessed: April 2026, 2024. [Online]. Available: https://docs.celeryq.dev

[29] ReportLab Inc., *Reportlab – pdf generation for python*, Accessed: April 2026, 2024. [Online]. Available: https://www.reportlab.com

[30] G. van Rossum, B. Warsaw, and N. Coghlan, *Pep 8 – style guide for python code*, Accessed: April 2026, 2001. [Online]. Available: https://peps.python.org/pep-0008/

[31] APNIC, *Dnssec validation statistics*, Accessed: April 2026, 2023. [Online]. Available: https://stats.labs.apnic.net/dnssec

[32] SURBL, *Surbl – multi-blocklist for uri reputation*, Accessed: April 2026, 2024. [Online]. Available: https://www.surbl.org

[33] OpenPhish, *Openphish – phishing intelligence*, Accessed: April 2026, 2024. [Online]. Available: https://openphish.com

[34] abuse.ch, *Urlhaus – malware url exchange*, Accessed: April 2026, 2024. [Online]. Available: https://urlhaus.abuse.ch

Chapter A

API Reference

This appendix lists all REST API endpoints exposed by the DNS Guard backend. All endpoints are served from the base URL `http://localhost:8000`. Requests and responses use JSON unless otherwise stated.

Table A.1: API Endpoint Reference

Method	Endpoint	Description
POST	`/analyze`	Full domain analysis. Accepts `{"domain": "example.com"}`. Returns DNS records, WHOIS, certs, passive DNS, typosquat matches, subdomains, reverse IP, threat intel, geolocation, threat score, verdict, and feature explanations.
POST	`/chat`	AI chatbot message. Accepts `{"domain": "...", "intent": "...", "message": "..."}`. Returns a text response from the LLM or rule-based fallback.
GET	`/report/{domain}/pdf`	Downloads a PDF report for the given domain. Triggers a fresh analysis and returns the result as an `application/pdf` file attachment.
GET	`/whois/{domain}`	Returns WHOIS registration data including registrar, creation date, expiry date, and name servers.

Method	Endpoint	Description
GET	`/certs/{domain}`	Returns certificate transparency data from crt.sh including certificate count, issuer list, and wildcard certificate flag.
GET	`/passive-dns/{domain}`	Returns historical IP resolution data and a fast-flux detection flag.
GET	`/typosquat/{domain}`	Returns a list of brand domains similar to the submitted domain with edit-distance similarity scores.
GET	`/subdomains/{domain}`	Returns a list of discovered subdomains from brute-force enumeration and certificate transparency logs.
GET	`/reverse-ip/{domain}`	Returns the resolved IP address and a list of other domains co-hosted on the same IP.
GET	`/threat-intel/{domain}`	Returns blocklist check results from Spamhaus DBL and PhishTank.

Chapter B

Machine Learning Model Details

B.1 Feature List

The threat scoring model uses a 40-dimensional feature vector extracted from the domain name and its DNS records.

Table B.1: ML Model Feature Vector (40 Features)

No.	Feature Name	Type	Description
0	`domain_length`	Integer	Total length of the full domain name string.
1	`sld_length`	Integer	Length of the second-level domain.
2	`subdomain_depth`	Integer	Number of dots in the domain.
3	`hyphen_count`	Integer	Number of hyphens in the domain name.
4	`sld_entropy`	Float	Shannon entropy of the SLD characters. High entropy suggests DGA-generated names.
5	`digit_ratio`	Float	Ratio of digit characters to total characters in the SLD.
6	`consonant_ratio`	Float	Ratio of consonant characters in the SLD.
7	`long_digit_seq`	Binary	1 if the SLD contains 4 or more consecutive digits.
8	`high_risk_tld`	Binary	1 if the TLD is in the high-risk list (e.g. `.tk`, `.xyz`, `.top`).
9	`brand_impersonation`	Binary	1 if the SLD contains the name of a well-known brand.
10	`phish_keywords`	Binary	1 if the SLD contains keywords such as `login`, `secure`, `verify`.

No.	Feature Name	Type	Description
11	`spam_keywords`	Binary	1 if the SLD contains keywords such as `free`, `win`, `prize`.
12	`has_a_record`	Binary	1 if the domain has at least one A record.
13	`has_aaaa_record`	Binary	1 if the domain has at least one AAAA record.
14	`has_mx_record`	Binary	1 if the domain has at least one MX record.
15	`ns_count`	Integer	Number of NS records for the domain.
16	`a_record_count`	Integer	Number of A records. A high count is a fast-flux indicator.
17	`has_soa`	Binary	1 if the domain has a SOA record.
18	`has_cname`	Binary	1 if the domain has a CNAME record.
19	`spf_present`	Binary	1 if a valid SPF record is found in TXT records.
20	`dmarc_present`	Binary	1 if a valid DMARC record is found in TXT records.
21	`dkim_present`	Binary	1 if a DKIM record is found in TXT records.
22	`spf_includes`	Binary	1 if the SPF record uses `include:` directives.
23	`fast_flux`	Binary	1 if the domain has more than 5 A records.
24	`many_ns`	Binary	1 if the domain has more than 6 NS records.
25	`no_mx_has_a`	Binary	1 if the domain has an A record but no MX record.
26	`dnskey_present`	Binary	1 if a DNSKEY record is present.
27	`rrsig_present`	Binary	1 if an RRSIG record is present.
28	`ds_present`	Binary	1 if a DS record is present.
29	`no_dnssec`	Binary	1 if neither DNSKEY nor RRSIG is present.
30	`mx_no_spf`	Binary	1 if the domain has MX records but no SPF record.
31	`mx_no_dmarc`	Binary	1 if the domain has MX records but no DMARC record.

No.	Feature Name	Type	Description
32	`very_long_sld`	Binary	1 if the SLD is longer than 20 characters.
33	`high_entropy_sld`	Binary	1 if the SLD entropy is greater than 3.5.
34	`deep_subdomain`	Binary	1 if the domain has more than 3 dots.
35	`brand_in_subdomain`	Binary	1 if a brand name appears in a subdomain but not in the SLD.
36–39	`reserved_36--39`	Float	Reserved for future features. Currently 0.0.

B.2 Training Data Summary

The model was trained on a labelled dataset compiled from the following sources:

- **Malicious domains:** Spamhaus DBL, PhishTank, and abuse.ch URLhaus feeds.
- **Legitimate domains:** Alexa top 1 million domain list, filtered to remove any domains present in threat feeds.

The dataset was balanced to contain an approximately equal number of malicious and legitimate samples.

B.3 Model Evaluation

Table B.2: ML Model Evaluation Metrics

Metric	Value
Accuracy	To be updated after final training
Precision	To be updated after final training
Recall	To be updated after final training
F1 Score	To be updated after final training
AUC-ROC	To be updated after final training

Chapter C

Sample Scan Output

The following is a representative sample of the JSON response returned by the `POST /analyze` endpoint for the domain `example.com`.


```
{
  "domain": "example.com",
  "threat_score": 12.5,
  "verdict": "Low Risk",
  "explanations": [
    { "feature": "no_dnssec", "value": 1.0, "contribution": 8.2 },
    { "feature": "mx_no_dmarc", "value": 1.0, "contribution": 4.3 }
  ],
  "dns_records": {
    "A": ["93.184.216.34"],
    "NS": ["a.iana-servers.net", "b.iana-servers.net"],
    "MX": [],
    "TXT": ["v=spf1 -all"],
    "SOA": ["ns.icann.org. noc.dns.icann.org. ..."]
  },
  "dnssec": { "dnskey": true, "rrsig": true, "ds": true },
  "whois": {
    "registrar": "ICANN",
    "creation_date": "1995-08-14",
    "expiry_date": "2025-08-13",
    "name_servers": ["A.IANA-SERVERS.NET", "B.IANA-SERVERS.NET"]
  },
  "certs": { "count": 4, "has_wildcard": false,
             "issuers": ["DigiCert Inc", "Let's Encrypt"] },
  "passive_dns": { "ips": ["93.184.216.34"], "fast_flux": false },
  "typosquat": { "matches": [] },
```

```
  "subdomains": { "found": ["www.example.com", "mail.example.com"] },
  "reverse_ip": {
    "ip": "93.184.216.34",
    "domains": ["example.com", "example.net", "example.org"]
  },
  "threat_intel": { "spamhaus_dbl": false, "phishtank": false },
  "geo": [{
    "ip": "93.184.216.34", "country": "United States",
    "city": "Norwell", "lat": 42.1596, "lon": -70.8217
  }]
}
```

Chapter D

Environment Variables and Configuration

All environment variables are defined in the `.env` file at the root of the project. A template is provided in `.env.example`.

Table D.1: Environment Variables and Configuration

Variable	Default Value	Description
POSTGRES_USER	dnsabuse	PostgreSQL database username.
POSTGRES_PASSWORD	changeme	PostgreSQL password. Change in production.
POSTGRES_DB	dnsabuse_db	Name of the PostgreSQL database.
DATABASE_URL	postgresql://...	Full SQLAlchemy connection string.
REDIS_URL	redis://redis:6379/0	Redis connection URL for Celery.
QDRANT_URL	http://qdrant:6333	URL of the Qdrant vector database.
AI_SERVICE_URL	http://ai:8001	Internal URL of the AI/ML service.
GROQ_API_KEY	(empty)	Groq API key for the AI chatbot. Get a free key at https://console.groq.com.
LLM_MODEL	llama-3.1-8b-instant	Groq model name for the chatbot.
VIRUSTOTAL_API_KEY	(empty)	Optional VirusTotal API key.
SHODAN_API_KEY	(empty)	Optional Shodan API key.

Variable	Default Value	Description
`SECRET_KEY`	`changeme`	App secret key. Must be changed in production.
`ENVIRONMENT`	`development`	Set to `production` to disable debug features.

Chapter E

Installation and Deployment Guide

E.1 Prerequisites

- **Docker Desktop** (v4.0+) – https://www.docker.com/products/docker-desktop
- **Git** – https://git-scm.com
- (Optional) **Groq API Key** – https://console.groq.com (free tier available)

E.2 Installation Steps

Step 1: Clone the repository

```
git clone <repository-url>
cd FYP
```

Step 2: Configure environment variables

```
cp .env.example .env
```

Open `.env` and change `POSTGRES_PASSWORD` and `SECRET_KEY` for any non-development use.

Step 3: (Optional) Configure Groq API for AI chatbot

Sign up at https://console.groq.com and create a free API key. Add it to `.env`:

```
GROQ_API_KEY=gsk_your_key_here
```

If no key is provided, the chatbot will use the built-in rule-based fallback engine.

Step 4: Build and start all services

```
docker compose up --build -d
```

This starts six containers: frontend, backend, AI service, PostgreSQL, Redis, and Qdrant. The first build may take several minutes.

Step 5: Open the application

```
http://localhost:3000
```

E.3 Stopping the Application

```
docker compose down
```

To also remove all stored data:

```
docker compose down -v
```

E.4 Updating Threat Feeds

```
python scripts/update_feeds.py
```

Chapter F

Sample Generated PDF Report

The following pages contain a sample PDF report generated by DNS Guard for the domain `nfsu.ac.in`. This report is produced by the `GET /report/{domain}/pdf` endpoint and is generated using the ReportLab library.

DNS Guard

Domain Threat Intelligence Report

18 /100

Low Risk

Domain: nfsu.ac.in

Generated: 2026-04-24 16:33 UTC

Risk Summary & Key Findings

MEDIUM	**No Dnssec** — No DNSSEC records found — DNS responses are not cryptographically signed
MEDIUM	**Mx No Dmarc** — MX record present but no DMARC — no email authentication policy

WHOIS Information

Registrar	ERNET India
Created	2020-10-15
Expires	2030-10-15
Domain age	2017 days
Days to expiry	1634 days
New domain	✗ No
Expiring soon	✗ No

DNS Records

A	117.239.177.124
MX	10 alt4.aspmx.l.google.com. 15 jhb6icnp62gombomdaogeqcoz6s2amnwr7tw4zdhqv636bw5udhq.mx-verification.google.com. 5 alt1.aspmx.l.google.com. 5 alt2.aspmx.l.google.com. 1 aspmx.l.google.com. 10 alt3.aspmx.l.google.com.
NS	ns-1127.awsdns-12.org. ns-1719.awsdns-22.co.uk. ns-35.awsdns-04.com. ns-604.awsdns-11.net.

TXT	"cisco-ci-domain-verification=537a0fbb308d574019f4d83e3d097d428744ef328c41c9f037e09325d5c135dc" "globalsign-domain-verification=0wq85be9X09hdNf2EhPDRc_oXvg7c31uXQX_L4pm6L" "google-site-verification=zsUfE6K5vqjrC_2KKD5PGjB46N9AlscmBmcv0Ze2vBM" "v=spf1 include:_spf.leewaysofttech.com include:_spf.google.com include:spf.mandrillapp.com include:spf.smtpconnect.com ~all" "Sendinblue-code:a8581e01263ce2669e381130c08a8625"
SOA	ns-604.awsdns-11.net. awsdns-hostmaster.amazon.com. 1 7200 900 1209600 86400

DNSSEC Status

✗ **DNSSEC Not Configured**

DNSKEY	✗ No
RRSIG	✗ No
DS	✗ No

Certificate Transparency

Total certificates	—
Wildcard certificates	—
Issued last 30 days	—
Certificate spike	✗ No
Has wildcards	✗ No

Passive DNS

Total records	5
Unique IPs seen	5
Fast-flux suspected	✗ No

IP Address	Seen
117.239.183.26	1
117.239.177.124	1
103.118.16.128	1
210.212.128.87	1
13.126.225.76	1

Typosquat Detection

■ **Possible typosquat detected**

Target Domain	Edit Distance
abc.es	1
1c.ru	1
as.com	1
ad.nl	1
lc.chat	1

Subdomain Enumeration

Total found	41
Suspicious	1

Suspicious subdomains:

- `login.nfsu.ac.in`

Reverse IP Lookup

Resolved IP	117.239.177.124
Domains on same IP	18
High-density host	✗ No

Co-hosted domains:

- `beta.nfsu.ac.in`
- `bhopal.nfsu.ac.in`
- `bhubaneswar.nfsu.ac.in`
- `chennai.nfsu.ac.in`
- `delhi.nfsu.ac.in`
- `dfeg.nfsu.ac.in`
- `dharwad.nfsu.ac.in`
- `goa.nfsu.ac.in`
- `gujarat.nfsu.ac.in`
- `guwahati.nfsu.ac.in`

Threat Intelligence

✓ **Not found on any blocklist**

Spamhaus DBL	Clean
PhishTank	Clean

Generated by DNS Guard · 2026-04-24 16:33 UTC · For security research purposes only.

List of Publications / Online References

Online References

1. P. Mockapetris, "Domain Names – Concepts and Facilities," RFC 1034, Internet Engineering Task Force, 1987. [Online]. Available: https://www.rfc-editor.org/rfc/rfc1034

2. R. Arends, R. Austein, M. Larson, D. Massey, and S. Rose, "DNS Security Introduction and Requirements," RFC 4033, IETF, 2005. [Online]. Available: https://www.rfc-editor.org/rfc/rfc4033

3. S. Kitterman, "Sender Policy Framework (SPF)," RFC 7208, IETF, 2014. [Online]. Available: https://www.rfc-editor.org/rfc/rfc7208

4. Anti-Phishing Working Group, "Phishing Activity Trends Report, Q4 2023," APWG, 2023. [Online]. Available: https://apwg.org/trendsreports/

5. VirusTotal, "Free Online Virus, Malware and URL Scanner." [Online]. Available: https://www.virustotal.com

6. MXToolbox, "Email Header Analyzer, MX Record Lookup." [Online]. Available: https://mxtoolbox.com

7. Shodan, "The Search Engine for the Internet of Everything." [Online]. Available: https://www.shodan.io

8. SecurityTrails, "DNS Intelligence and Historical Data." [Online]. Available: https://securitytrails.com

9. HackerTarget, "DNSdumpster – DNS Recon and Research." [Online]. Available: https://dnsdumpster.com

10. PhishTank, "Free Community Site for Phishing Data." [Online]. Available: https://www.phishtank.com

11. The Spamhaus Project, "Domain Block List (DBL)." [Online]. Available: https://www.spamhaus.org

12. SURBL, "Multi-Blocklist for URI Reputation." [Online]. Available: https://www.surbl.org

13. OpenPhish, "Phishing Intelligence." [Online]. Available: https://openphish.com

14. abuse.ch, "URLhaus – Malware URL Exchange." [Online]. Available: https://urlhaus.abuse.ch

15. APNIC, "DNSSEC Validation Statistics," 2023. [Online]. Available: https://stats.labs.apnic.net/dnssec

16. Vercel, "Next.js – The React Framework for the Web." [Online]. Available: https://nextjs.org

17. Tailwind Labs, "Tailwind CSS – A Utility-First CSS Framework." [Online]. Available: https://tailwindcss.com

18. S. Ramirez, "FastAPI – Modern, Fast Web Framework for Building APIs with Python." [Online]. Available: https://fastapi.tiangolo.com

19. Ask Solem et al., "Celery – Distributed Task Queue." [Online]. Available: https://docs.celeryq.dev

20. The PostgreSQL Global Development Group, "PostgreSQL – The World's Most Advanced Open Source Relational Database." [Online]. Available: https://www.postgresql.org

21. Redis Ltd., "Redis – The Real-Time Data Platform." [Online]. Available: https://redis.io

22. Qdrant, "Vector Database for the Next Generation of AI Applications." [Online]. Available: https://qdrant.tech

23. Groq Inc., "Groq API – Fast AI Inference." [Online]. Available: https://console.groq.com

24. B. Halley and B. Westenberg, "dnspython – A DNS Toolkit for Python." [Online]. Available: https://www.dnspython.org

25. ReportLab Inc., "ReportLab – PDF Generation for Python." [Online]. Available: https://www.reportlab.com

26. Docker Inc., "Docker – Accelerated Container Application Development." [Online]. Available: https://www.docker.com

27. G. van Rossum, B. Warsaw, and N. Coghlan, "PEP 8 – Style Guide for Python Code," 2001. [Online]. Available: https://peps.python.org/pep-0008/

28. ICANN, "ICANN – Internet Corporation for Assigned Names and Numbers." [Online]. Available: https://www.icann.org

29. ICANN, "DNS Abuse – ICANN's Role and Efforts to Address DNS Abuse." [Online]. Available: https://www.icann.org/dns-abuse

30. ICANN, "Registration Data Access Protocol (RDAP)." [Online]. Available: https://www.icann.org/rdap

31. DNS Abuse Institute, "NetBeacon – Reporting DNS Abuse." [Online]. Available: https://dnsabuseinstitute.org

32. ICANN Security and Stability Advisory Committee (SSAC), "SAC115 – SSAC Advisory on DNS Abuse," ICANN, 2021. [Online]. Available: https://www.icann.org/en/system/files/files/sac-115-en.pdf

33. M3AAWG and ICANN, "DNS Abuse Mitigation – Best Practices for Registrars and Registries," 2022. [Online]. Available: https://www.icann.org/en/blogs/details/dns-abuse-mitigation

Publications

No publications have been produced from this work at the time of submission. The project is an academic final year dissertation submitted to the National Forensic Sciences University, Delhi Campus, in partial fulfilment of the B.Tech.-M.Tech. degree in Computer Science and Engineering (Cybersecurity).

Rupam Plagg File.pdf

ORIGINALITY REPORT

5%	3%	1%	2%
SIMILARITY INDEX	INTERNET SOURCES	PUBLICATIONS	STUDENT PAPERS

PRIMARY SOURCES

1	Submitted to National Forensic Sciences University Student Paper	2%
2	ijsrem.com Internet Source	<1%
3	myfik.unisza.edu.my Internet Source	<1%
4	Submitted to Middlesex University Student Paper	<1%
5	Submitted to Quest International University Student Paper	<1%
6	Angus Wong. "Network Infrastructure Security ' Address Configuration and Naming", Network Infrastructure Security, 2009 Publication	<1%
7	Submitted to Charles Sturt University Student Paper	<1%
8	Submitted to Panipat Institute of Engineering & Technology Student Paper	<1%
9	www.mdpi.com Internet Source	